

JAVA

01

Core Java

Build the solid Java foundation every full-stack developer needs — from your first class to concurrent threads.

DURATION	LEVEL	MODULE	UPDATED
18 Sessions	Beginner	Module 1	June 2026

IN THIS MODULE

01	Introduction to Java & Environment Setup
02	Data Types, Variables & Operators
03	Conditional Statements
04	Loops & Flow Control
05	Arrays
06	Introduction to Object-Oriented Programming
07	Classes, Objects & Methods
08	Constructors & Method Overloading
09	Inheritance & Reusability
10	Polymorphism
11	Encapsulation & Abstraction
12	String Handling
13	Exception Handling – Basics
14	Exception Handling – Advanced
15	File I/O

16 Collection Framework – Part 1

17 Collection Framework – Part 2

18 Threads & Recap

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to Java & Environment Setup

◆ Learning Objectives

- Explain Java's origin, evolution, and why it dominates enterprise and Android development
- Distinguish between JVM, JRE, and JDK and describe the role of each component
- Install JDK 17 LTS and configure Eclipse or IntelliJ IDEA Community Edition
- Write, compile, and run a first Java program from the IDE
- Identify the structural elements of a Java class: class keyword, main method, and `System.out.println()`

01 What is Java & Why It Matters

Java was created in 1991 by James Gosling at Sun Microsystems under the codename 'Oak' and released publicly in 1995. Today it underpins large-scale Indian systems — Paytm's payment engine, IRCTC's ticket booking backend, and most Android apps on your phone are all built with Java. With over 3 billion devices running the JVM globally, learning Java opens the widest range of software engineering careers available in India and abroad.

CONCEPT — Write Once, Run Anywhere (WORA)

Java compiles your source code into platform-neutral bytecode (.class files) rather than OS-specific machine code. The JVM installed on any machine — a Windows laptop in Ahmedabad, a Linux server in Mumbai, or an Android phone in Surat — runs that same bytecode identically. This is why one Java application can be deployed across an entire enterprise without rewriting it for each operating system.

JAVA GLOBAL FOOTPRINT

3 Billion+

Source: Oracle — devices running Java worldwide (2023)

02 JVM, JRE & JDK — The Java Ecosystem

Component	Full Name	Who Uses It	What It Provides
JVM	Java Virtual Machine	Everyone (embedded)	Bytecode interpreter + garbage collector
JRE	Java Runtime Environment	End-users running apps	JVM + standard class libraries

Component	Full Name	Who Uses It	What It Provides
JDK	Java Development Kit	Developers — install this	JRE + javac compiler + debugger + tools

CONCEPT — From Source File to Running Program

You write `HelloWallet.java` -> `javac` compiles it into `HelloWallet.class` (bytecode) -> the JVM reads the bytecode and translates it to native machine instructions via its JIT (Just-In-Time) compiler. The JVM also runs garbage collection automatically, reclaiming memory from objects you no longer use — no manual `free()` calls like in C.

03 Installing Java & Your IDE

Download JDK 17 LTS from adoptium.net (the free Temurin build — recommended) or oracle.com/java. For your IDE install Eclipse IDE for Java Developers or IntelliJ IDEA Community Edition — both are free and widely used in Indian IT companies. After installation, always verify the setup in your terminal before writing any code.

TRY THIS LIVE

Open Terminal (Mac/Linux) or Command Prompt (Windows) and run both commands below. Each should print a version number. If `'javac'` is not found, your `PATH` does not include the JDK bin folder — add it via Environment Variables on Windows or `~/.bashrc` on Linux/Mac, then reopen the terminal.

```
java -version           Checks that the JVM (runtime) is installed
javac -version          Checks that the Java compiler (JDK) is installed
# Expected: java 17.0.x and javac 17.0.x  Any 17.x version means you are ready to code
```

04 Anatomy of a Java Class

Every Java program lives inside a class. The source filename must exactly match the public class name — capitalisation included, because Java is case-sensitive. The `main()` method is the single entry point: the JVM always looks here to begin execution. `System.out.println()` prints a line followed by a newline; `System.out.print()` prints without a newline.

```
// File: HelloDigitalWallet.java
public class HelloDigitalWallet {
    public static void main(String[] args) {
```

Filename must match the class name exactly
Class declaration — use PascalCase
JVM entry point — exact signature required

```
System.out.println("Hello Digital Wallet!");
```

Prints text + newline to console

```
System.out.println("Balance: " + 5000);
```

String + int concatenation with + operator

```
}  
}
```

Close main

Close class

PRO TIP — Java Naming Conventions

Classes use PascalCase (HelloDigitalWallet). Methods and variables use camelCase (checkBalance, walletId). Constants use ALL_CAPS_WITH_UNDERSCORES (MAX_TRANSFER_LIMIT). Following these from day one makes your code readable to every Java developer, including senior engineers who review GitHub profiles during hiring at companies like Infosys, TCS, or Wipro.

CASE STUDY — Paytm: Java Powering India's Largest Digital Wallet

Context — Paytm (One97 Communications), based in Noida, is India's largest digital payments platform serving 300 million+ users.

Challenge — Handle hundreds of millions of daily UPI transactions, wallet top-ups, and merchant payments with sub-second latency and near-zero downtime.

What they did — Paytm's core transaction engine is written in Java. The strong type system catches category-level bugs at compile time before they reach production. The JVM's automatic garbage collection manages memory across thousands of concurrent threads, so engineers focus on payment logic rather than low-level memory management.

Result — Paytm scales reliably to crore-level peak transactions during Diwali, deployed as JVM-based microservices on cloud infrastructure.

Takeaway — The class declarations, main method, and JVM lifecycle you practise today are the exact same foundations that move thousands of crores of rupees every hour.

Source: Paytm Engineering Blog, medium.com/paytm-engineering (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lecture 1: Introduction (Prof. Debasis Samanta, IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You can now write and run a basic Java program. Next, learn how Java stores and works with different kinds

of data.

Explore deeper — Oracle Java SE 17 Getting Started — docs.oracle.com/javase/17/tutorial/getStarted/

Try on your own — Modify HelloDigitalWallet to print your name, city, and a dummy wallet balance stored in a variable — all on separate lines.

Connect the dots — Session 02 covers primitive data types and operators — the tools you need to store a balance and calculate interest.

Going further — Research how the JVM's JIT compiler identifies 'hot' bytecode paths and compiles them to native machine instructions for near-native performance.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java tutor. Explain the difference between JVM, JRE, and JDK in simple Hinglish using the analogy of a chai shop: JVM = the stove, JRE = stove + all ingredients, JDK = stove + ingredients + the recipe book + the chef. Keep it under 150 words.

SESSION

02

60 MIN · FUNDAMENTALS

Data Types, Variables & Operators

◆ Learning Objectives

- List all 8 Java primitive data types with their size and default value
- Declare and initialise variables following Java naming conventions
- Apply arithmetic, relational, logical, and ternary operators correctly
- Perform both widening (implicit) and narrowing (explicit) type casts
- Write a program that calculates simple interest on a digital wallet balance

01 Primitive Data Types

Java has exactly 8 built-in primitive types that store values directly in memory — not as objects. Choosing the right type matters for both correctness and performance. In fintech systems like HDFC's internet banking or PhonePe's UPI layer, using long for transaction amounts prevents integer overflow that would produce wildly incorrect figures.

Type	Size	Range / Use	Declaration Example
byte	1 byte	-128 to 127 — tiny counts	byte retryCount = 3;
short	2 bytes	-32,768 to 32,767 — small integers	short year = 2024;
int	4 bytes	~±2.1 billion — general integers	int balance = 50000;
long	8 bytes	Very large integers — use for amounts, IDs	long txnId = 9876543210L;
float	4 bytes	Decimal, 6-7 digits precision	float rate = 3.5f;
double	8 bytes	Decimal, 15 digits precision (default)	double interest = 1234.56;
char	2 bytes	Single Unicode character	char grade = 'A';
boolean	~1 bit	true / false only	boolean isActive = true;

CONCEPT — Primitive vs Non-Primitive

Primitives (int, double, boolean...) store the actual value directly in the variable's memory slot. Non-primitives (String, arrays, objects) store a reference — a memory address pointing to where the data lives on the heap. This is why String is capitalised (it is a class) while int is not. Passing a primitive to a method copies the value;

passing an object copies only the reference.

02 Variables — Declaration, Initialisation & Scope

A variable is a named storage location with a fixed type. You must declare the type before using the variable. Think of a variable as a labelled column in a database row — the data type is fixed (like a column's VARCHAR or INT), but the value can change. Local variables have no default value and must be initialised before use; instance variables get type-specific defaults (0, 0.0, false, null).

```
int walletBalance = 5000;
```

Declare int and initialise in one step

```
double interestRate;
```

Declare without value — default is 0.0 for instance vars

```
interestRate = 3.5;
```

Assign value later

```
final double GST_RATE = 0.18;
```

final = constant — cannot be changed after assignment

```
String ownerName = "Priya Patel";
```

String is a class, not a primitive — note capital S

```
var txnCount = 10;
```

var (Java 10+) infers type from right side — int here

CONCEPT — Variable Scope

A local variable declared inside a method exists only within that method block. An instance variable declared in a class but outside any method belongs to each object separately. A static variable belongs to the class itself, shared across all objects. Mixing up scope is one of the most common beginner compile errors — 'cannot find symbol' usually means you used a variable outside its scope.

03 Operators in Java

Java operators work on one or more operands and produce a result. Arithmetic operators perform math. Relational operators compare two values and always return a boolean. Logical operators combine booleans. The compound assignment operators (+=, -=) are shorthand that also improve readability.

Category	Operators	Example	Result
Arithmetic	+ - * / % ++ --	balance + 1000	New sum
Relational	== != > < >= <=	balance >= 100	true or false
Logical	&& !	isActive && balance > 0	true or false
Assignment	= += -= *= /= %=	balance += 500;	balance = balance + 500

Category	Operators	Example	Result
Ternary	condition ? a : b	balance > 0 ? "Active" : "Empty"	String result

TRY THIS LIVE

Create a class called WalletCalc. Declare: int principal = 10000; double rate = 5.0; int time = 2; Compute SI = (principal * rate * time) / 100. Print the result. Then use the ternary operator to print 'Profitable' if SI > 800, else 'Low return'. Run it and verify the output.

04 Type Casting

Type casting converts a value from one type to another. Widening (implicit) casting happens automatically when moving to a larger type — safe, no data loss. Narrowing (explicit) casting requires you to write the target type in parentheses — may lose the decimal portion or overflow. Always narrow deliberately; accidental narrowing is a common source of financial calculation bugs.

```
int txnCount = 150;
double avgAmount = txnCount;
double price = 99.99;
int rounded = (int) price;
// Simple Interest demo
int principal = 10000; double rate = 5.0; int time = 2;
    Declare wallet interest inputs
double si = (principal * rate * time) / 100;
System.out.printf("Interest: %.2f\n", si);
```

int variable
Implicit widening: int -> double, safe
double variable
Explicit narrowing: truncates to 99, NOT rounded
Result: 1000.0
printf formats to 2 decimal places

Widening (Implicit)

byte -> short -> int -> long -> float -> double. Java does this automatically. No data loss. Use freely when moving up the type hierarchy.

Narrowing (Explicit)

double -> int, long -> int. You must write (int) explicitly. Decimal part is truncated, not rounded. Always verify the result is what you expect.

CASE STUDY — IRCTC: Why Data Types Matter in Ticket Booking

Context — IRCTC processes over 1 million online bookings per day, making it one of the world's busiest e-commerce platforms.

Challenge — Store PNR numbers (10-digit), passenger counts, fare amounts, and train codes accurately without overflow or precision errors across a nationwide system.

What they did — The Java backend uses long for PNR numbers (10-digit values exceed int's ~2.1 billion

limit), double for fare calculations, and int for seat counts. Using int for a PNR — a common beginner mistake — would corrupt data for bookings beyond the int range.

Result — Accurate PNR generation and fare calculation with no overflow errors, even during Tatkal booking surges when lakhs of users hit the system simultaneously.

Takeaway — Choosing the right data type is not academic. It is the difference between a correct booking confirmation and silent data corruption that costs the organisation — and passengers — money.

Source: IRCTC Annual Report 2022-23; Java architecture — publicly discussed at NIC tech conferences

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 3-4: Data Types, Variables and Operators (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can store and compute with Java data. Next, use that data to make decisions in your programs.

Explore deeper — Java Operators tutorial —
docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html

Try on your own — Build a 'Currency Converter': take an int rupees amount, multiply by a double exchange rate, and print the USD result formatted to 2 decimal places using `System.out.printf()`.

Connect the dots — Session 03 introduces conditional statements — you will apply relational and logical operators to control which code path runs.

Going further — Look up Java's `BigDecimal` class — used in fintech systems where double's floating-point imprecision makes it unsuitable for currency arithmetic.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java coding mentor. Explain the difference between int, long, float, and double in simple Hinglish using the example of storing a UPI transaction amount in a digital wallet. Tell me which type to use and why. Keep it under 150 words.

SESSION

03

60 MIN · CONTROL FLOW

Conditional Statements

◆ Learning Objectives

- Write if, if-else, and if-else-if chains to handle multiple decision paths
- Use switch statements for multi-branch logic based on discrete values
- Implement nested conditions to model real-world multi-factor decisions
- Apply the ternary operator for compact single-expression conditionals
- Map conditional logic to the concept of WHERE clause filtering in SQL

01 Why Programs Need to Make Decisions

Every useful program makes decisions: should this transaction go through? Is this user eligible for cashback? Which wallet tier applies? In Java, conditional statements let your program evaluate a boolean expression and choose which block of code to execute. Every condition evaluates to exactly true or false — there is no 'maybe' in Java.

CONCEPT — Boolean Logic is the Gate

Every condition inside an if statement must resolve to a boolean. Relational operators (>, <, ==, !=, >=, <=) compare values and return boolean. Logical operators (&& = AND, || = OR, ! = NOT) combine booleans. Think of && as 'both conditions must be true' and || as 'at least one must be true' — exactly like WHERE clauses with AND / OR in SQL.

02 if, if-else, and if-else-if

The if statement executes a block only when the condition is true. The else block executes when it is false. Chaining else-if lets you handle multiple mutually-exclusive cases in order — Java checks each condition top to bottom and executes only the first true branch.

```
int balance = 80;
if (balance < 100) {
    System.out.println("Low Balance Warning!");
    Executes only when condition is true
} else if (balance < 500) {
    System.out.println("Balance is getting low.");
    Checked only if the first condition was false
} else {
    System.out.println("Balance OK.");
    Runs if none of the above conditions matched
}
    Close if block
```

Wallet balance in rupees

Check if balance is critically low

CONCEPT — Mapping Conditionals to SQL WHERE

A Java `if (balance >= minBalance && isActive)` mirrors a SQL `WHERE balance >= min_balance AND is_active = 1`. The boolean logic is identical — the difference is that Java acts on objects in memory while SQL filters rows in a table. Understanding this connection makes switching between backend Java code and database queries intuitive.

03 switch Statement

When you need to match one variable against many possible discrete values, `switch` is cleaner than a long `if-else-if` chain. Java 14+ supports `switch` expressions (arrow syntax) that eliminate fall-through bugs. Use `switch` when the variable is an `int`, `char`, `String`, or `enum`.

```
String paymentMode = "UPI";
switch (paymentMode) {
    case "UPI":
        System.out.println("Processing UPI payment");
        break;
    case "CARD":
        System.out.println("Processing card payment");
        break;
    case "WALLET":
        System.out.println("Deducting from wallet");
        break;
    default:
        System.out.println("Unknown payment mode");
}
```

Variable to switch on

Match paymentMode against cases

Matches 'UPI'

`break` exits the switch — without it, fall-through occurs

Runs if no case matched — always include this

Close switch

if-else-if

Better when conditions involve ranges, computed values, or multiple variables (e.g., `balance > 1000 && isVerified`). More flexible but verbose.

switch

Better when one variable is matched against a fixed list of known values (e.g., `paymentMode == 'UPI' / 'CARD' / 'WALLET'`). Cleaner and faster for many discrete cases.

04 Ternary Operator & Nested Conditions

The ternary operator packs a simple `if-else` into one expression: `condition ? valueIfTrue : valueIfFalse`. Use it when you need to assign a value based on a condition. Nested conditions (`if` inside `if`) model multi-factor decisions — for example, checking both account tier and balance before granting cashback. Keep nesting to 2 levels maximum for readability.

```
// Ternary example
int balance = 1500;
String status = (balance >= 1000) ? "Premium" : "Basic";
    One-line conditional assignment
System.out.println("Tier: " + status);           Prints: Tier: Premium

// Nested condition — cashback eligibility
boolean isVerified = true;
if (isVerified) {                               Outer check — is the account KYC-verified?
    if (balance >= 1000) {                       Inner check — sufficient balance?
        System.out.println("Cashback: Rs. 50 applied");
    } else {
        System.out.println("Top up Rs." + (1000 - balance) + " for cashback");
    }
}
```

TRY THIS LIVE

Write a program that declares `int balance` and `String kycStatus`. Use nested if-else to print: 'Cashback eligible' if `balance > 500` AND `kycStatus` equals 'VERIFIED'; 'KYC pending' if balance is fine but KYC is not done; 'Low balance' if balance is the problem. Test with different combinations.

PRO TIP — `==` vs `.equals()` for Strings

Never use `==` to compare String values. `==` checks whether two variables point to the exact same object in memory. `.equals()` checks whether the content is the same. Use `paymentMode.equals("UPI")` not `paymentMode == "UPI"`. This is a top interview question at every Java job and a common production bug in beginners' code.

CASE STUDY — PhonePe: Conditional Logic Driving UPI Routing

Context — PhonePe, based in Bengaluru, processes over 5 billion UPI transactions per month — one of the highest volumes globally.

Challenge — Route each payment to the correct bank channel, apply the right cashback rule, and display the right status — all in under 500 milliseconds.

What they did — The transaction engine uses layered conditional logic: first check payment mode (UPI / card / wallet), then check bank availability, then apply cashback rules based on merchant category and user tier. These nested if-else chains and switch blocks together determine the entire payment flow.

Result — PhonePe achieves 99.99% uptime and routes over 400 million daily transactions accurately, with cashback correctly applied for eligible users.

Takeaway — The if-else chains and switch blocks you are writing today are, in more complex form, the exact same logic driving the payments screen on 300 million Indian phones.

Source: PhonePe Engineering Blog, engineering.phonepe.com (2023)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 5-6: Control Flow and Conditionals (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now control which code path runs based on data. Next, you will repeat code blocks efficiently using loops.

Explore deeper — Java Control Flow — docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html

Try on your own — Build a 'Wallet Access Controller': ask the user to enter a PIN (use Scanner). If PIN == 1234, print 'Access Granted'. If wrong, print 'Wrong PIN — X attempts remaining' (decrement a counter). If 3 wrong attempts, print 'Account Locked'.

Connect the dots — Session 04 covers for, while, and do-while loops — you will combine them with conditionals to process lists of transactions.

Going further — Research Java's enhanced switch expression (Java 14+) with the arrow -> syntax that eliminates fall-through and returns values directly.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java instructor. Explain the difference between if-else and switch in simple Hinglish. Give an example based on a UPI payment app where one works better than the other. Tell me when to use .equals() vs == for String comparison and why. Keep it under 150 words.

SESSION

04

60 MIN · CONTROL FLOW

Loops & Flow Control

◆ Learning Objectives

- Write for, while, and do-while loops and choose the right one for each scenario
- Use break to exit a loop early and continue to skip the current iteration
- Build nested loops to process two-dimensional data
- Simulate a monthly transaction summary using a loop
- Connect loop iteration to the concept of traversing database records

01 Why Programs Need Loops

Repeating the same operation hundreds or millions of times is exactly what computers are built for — and exactly what makes software powerful. Calculating interest for every account in a bank, sending notifications to every registered user, or printing 10 transaction rows from a wallet history all require loops. Without loops, you would have to write each operation manually — impossible at scale.

CONCEPT — Loops Mirror Iterating Over Database Records

When a Java loop processes each element of an array or list, it is doing the same logical work as a SELECT statement iterating over every row returned by a query. The loop variable (i, txn, item) corresponds to the 'current row' — you process it, then move to the next. This mental model helps you switch between backend Java code and SQL queries intuitively.

02 for Loop

The for loop is the best choice when you know exactly how many times to repeat. It has three parts in its header: initialisation (runs once), condition (checked before each iteration), and update (runs after each iteration). Use it when working with arrays, index-based access, or fixed counts.

```
// Print a 6-month transaction summary
double[] monthlySpend = {4200, 3800, 5100, 4700, 6200, 3900};
    Array of 6 months' spending
double total = 0;                                Accumulator variable
for (int i = 0; i < monthlySpend.length; i++) {
    i starts at 0, increments each time
    System.out.println("Month " + (i+1) + ": Rs." + monthlySpend[i]);
    Print month and amount
    total += monthlySpend[i];                    Add to running total
}
```

```
}
System.out.println("Total spend: Rs." + total);
Print grand total after loop
```

03 while Loop & do-while Loop

Use while when you do not know the count in advance — the loop runs as long as a condition is true. Use do-while when the body must execute at least once before checking the condition — for example, showing a PIN entry prompt at least once regardless of the initial state.

```
// while: keep asking until balance is sufficient
int balance = 200; int required = 500;
while (balance < required) {           Condition checked BEFORE each iteration
    System.out.println("Top up needed. Current: " + balance);
    balance += 100;                   Simulate adding money
}
System.out.println("Sufficient balance: " + balance);

// do-while: show menu at least once
int choice;
do {                                   Body executes FIRST, condition checked AFTER
    System.out.println("1-Check Balance  2-Transfer  3-Exit");
    Menu always appears at least once
    choice = new java.util.Scanner(System.in).nextInt();
    Read user choice
} while (choice != 3);                Repeat until user chooses Exit
```

while Loop

Condition checked BEFORE body runs. If condition is false from the start, the body never executes. Use when the count is unknown and zero iterations is a valid outcome.

do-while Loop

Body runs FIRST, condition checked AFTER. Guarantees at least one execution. Use for menus, PIN prompts, or any 'do this then ask' scenario.

04 break, continue & Nested Loops

break immediately exits the loop — useful for stopping a search once you find what you need. continue skips the rest of the current iteration and jumps to the next — useful for filtering. Nested loops use one loop inside another: the outer loop runs once per row, the inner loop runs once per column — perfect for processing 2D data like a transaction grid.

```
// break: find first failed transaction
int[] txnAmounts = {200, 500, -1, 300, 100};    -1 marks a failed transaction
for (int i = 0; i < txnAmounts.length; i++) {
    if (txnAmounts[i] == -1) {
        System.out.println("Failed txn at index: " + i);
        break;                                Stop loop immediately after finding first failure
    }
}

// continue: skip negative amounts
for (int amt : txnAmounts) {                  Enhanced for-each loop
    if (amt < 0) continue;                    Skip this iteration for negative amounts
    System.out.println("Valid: " + amt);      Only prints non-negative amounts
}
```

TRY THIS LIVE

Create an array of 10 transaction amounts (mix of positives and a few -1 for failed). Use a for loop to: (a) skip failed transactions with continue, (b) sum all valid transactions, (c) break as soon as the running total exceeds Rs. 2000. Print the final total and the index where you stopped.

PRO TIP — Enhanced for-each Loop

Java's enhanced for loop (for (int amt : amounts)) is cleaner when you do not need the index. It reads: 'for each amt in amounts'. Use it for read-only traversal of arrays and collections. Use the indexed for loop only when you need to modify elements or need the position — for example, to update txnAmounts[i] inside the loop.

CASE STUDY — Zepto: Loops Powering Real-Time Inventory Checks

Context — Zepto, the Mumbai-based 10-minute grocery delivery startup, processes thousands of orders simultaneously across dark stores in Ahmedabad, Surat, and other cities.

Challenge — For every order placed, check stock availability for each item in the cart, calculate the total, and flag out-of-stock items — all in milliseconds.

What they did — The order processing service uses a for-each loop to iterate over every item in the cart list. Inside the loop, a conditional checks inventory count. If any item is out of stock, the loop flags it and uses continue to still process the remaining items. A break-like short-circuit exits early if the cart is empty.

Result — Zepto achieves sub-second cart validation for orders with 20-30 items, enabling the 10-minute delivery promise across 25+ cities.

Takeaway — The loop patterns — for-each to iterate, continue to skip, break to stop early — are the skeleton of every list-processing feature in any e-commerce or fintech application.

Source: Zepto Engineering interviews, Inc42 Tech coverage (2023)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 7-8: Loops and Flow Control (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now repeat operations efficiently. Next, learn how to store multiple values in a structured way using arrays.

Explore deeper — Java Loops Tutorial — docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html

Try on your own — Write a multiplication table printer: use nested loops so the outer loop goes 1 to 5 (tables) and the inner loop goes 1 to 10 (multiples). Print in a neat grid format.

Connect the dots — Session 05 introduces arrays — the data structure that loops are most frequently used with in real programs.

Going further — Research Java Streams (Java 8+) — a functional alternative to loops that uses `filter()`, `map()`, and `reduce()` to process collections declaratively.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java tutor. Explain the difference between for, while, and do-while loops in simple Hinglish. Use the analogy of standing in a bank queue: for = you know how many people are ahead; while = you don't know but keep checking; do-while = you walk in first and then check if you need to stay. Keep it under 150 words.

SESSION

05

60 MIN · DATA STRUCTURES

Arrays

◆ Learning Objectives

- Declare, initialise, and access single-dimensional arrays in Java
- Traverse arrays using both indexed for loops and enhanced for-each loops
- Declare and work with two-dimensional arrays for tabular data
- Build a program that stores and displays the last 10 wallet transactions
- Explain how arrays model a single table row or result set from a database query

01 What is an Array & Why It Matters

An array is a fixed-size, ordered container that stores multiple values of the same type under a single variable name. Instead of declaring `int txn1, txn2, txn3... up to txn100`, you declare `int[] transactions = new int[100]`. Arrays are the simplest form of data structure and the foundation for understanding lists, queues, and database result sets.

CONCEPT — Arrays Mirror Database Table Rows

A single-dimensional array is like one column of a database table — an ordered list of values of the same type. The array index (0, 1, 2...) maps to the row number. A two-dimensional array is like the full table — rows × columns. This is why, in Java backend code, you often convert a `ResultSet` from JDBC into an array or list for processing.

02 Declaring & Initialising Arrays

Array declaration tells Java the type and name. Initialisation allocates memory for a fixed number of elements. You can declare and initialise separately, or combine them. All elements start with the type's default value: 0 for `int`, 0.0 for `double`, null for `String`. Java arrays are zero-indexed — the first element is always at index 0.

```
// Method 1: Declare then allocate
int[] transactions;           Declare reference — no memory yet
transactions = new int[10];   Allocate space for 10 integers (all start at 0)

// Method 2: Declare + allocate in one line
double[] balances = new double[5];   5 doubles, each initialised to 0.0

// Method 3: Initialise with values
String[] modes = {"UPI", "CARD", "WALLET", "NETBANKING"};
// Length inferred as 4
```

```
// Assign and access
transactions[0] = 500;
transactions[9] = 1200;
System.out.println(transactions[0]);
```

Set first element (index 0)
Set last element (index length-1)
Read: prints 500

WARNING — `ArrayIndexOutOfBoundsException`

Accessing an index that does not exist (e.g., `transactions[10]` on a 10-element array) throws `ArrayIndexOutOfBoundsException` at runtime — one of the most common Java errors. Always use `i < array.length` in your loop condition, never `i <= array.length`. Validate user-supplied indices before using them.

03 Traversing Arrays with Loops

The standard loop for arrays is the indexed for loop when you need the position, and the enhanced for-each when you just need each value. Use `.length` (not `.length()`) to get the array size — arrays are not objects with methods, `.length` is a built-in field.

```
int[] txnAmounts = {500, 1200, 300, 850, 200, 400, 900, 150, 600, 750};
    Last 10 wallet transactions

// Indexed loop — use when you need i
for (int i = 0; i < txnAmounts.length; i++) {    i goes 0 to 9
    System.out.println("Txn " + (i+1) + ": Rs." + txnAmounts[i]);
}

// Enhanced for-each — clean when you only need the value
int total = 0;
for (int amt : txnAmounts) {                    Reads as 'for each amt in txnAmounts'
    total += amt;                                Accumulate
}
System.out.println("Total spend: Rs." + total);
    Prints sum of all 10 transactions
```

04 Multi-Dimensional Arrays

A 2D array stores data in a grid of rows and columns — think of it as a spreadsheet or a table. You declare it with two sets of brackets: `int[][]` grid. Access elements with two indices: `grid[row][col]`. Use nested loops to traverse: outer loop for rows, inner loop for columns.

```
// 2D array: 3 users × 4 months of spending
double[][] spending = {
    {4200, 3800, 5100, 4700},
    {2200, 2900, 3100, 2700},
    {6100, 5800, 7200, 6400}
};

// Row = user, Column = month
// User 0 (Ankit)
// User 1 (Priya)
// User 2 (Ravi)

for (int row = 0; row < spending.length; row++) {
    Iterate users
    double userTotal = 0;
    for (int col = 0; col < spending[row].length; col++) {
        Iterate months
        userTotal += spending[row][col];
    }
    System.out.println("User " + row + " total: Rs." + userTotal);
}
```

TRY THIS LIVE

Store the last 10 wallet transaction amounts in an `int[]` array. Write a program that: (1) prints all 10 transactions, (2) finds and prints the highest transaction, (3) finds and prints the lowest transaction, (4) prints the average. Use separate loops or a single loop — your choice.

COMMON INTERVIEW QUESTION

Top 5

Array reversal, find max/min, binary search, and two-sum appear in the top 5 most common Java interview questions — Glassdoor/LeetCode reports 2023

CASE STUDY — Amazon India: Array-Based Search for Lightning Deals

Context — Amazon India's Great Indian Festival sale sees millions of simultaneous searches for discounted products across categories.

Challenge — Store product prices for flash deals and find the minimum price in a category in microseconds during peak traffic.

What they did — Flash deal price lists are loaded into `int` or `double` arrays in memory (not fetched from DB per request). A simple linear scan — like the loop you just wrote — finds the minimum price. For large sorted lists, binary search reduces the comparisons from N to $\log(N)$.

Result — Amazon India serves deal searches in under 50 ms during peak sale traffic, partly because in-memory array lookups are orders of magnitude faster than repeated database queries.

Takeaway — The min-finding loop you just wrote is, at its core, the same algorithm powering 'lowest price' features in every e-commerce app used by crores of Indians daily.

Source: Amazon Engineering Blog, developer.amazon.com (2023)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 9-10: Arrays and Array Operations (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can store and process multiple values. Now learn how to model real-world entities as objects, not just data.

Explore deeper — Java Arrays Tutorial — docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

Try on your own — Write a 'Transaction Report': ask the user (using Scanner) to enter 5 transaction amounts. Store them in an array. Print all amounts, the total, average, and whether each transaction is above or below the average.

Connect the dots — Session 06 introduces Object-Oriented Programming — instead of isolated arrays of amounts, you will model a full Transaction object with amount, date, and type together.

Going further — Explore `java.util.Arrays` — it provides ready-made `sort()`, `binarySearch()`, and `fill()` methods so you do not have to implement them from scratch.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java tutor. Explain arrays in simple Hinglish using the analogy of a row of lockers at a railway station cloak room. Each locker has a number (index) and holds one item. Explain how declaring an array is like booking N lockers, and how you access a locker with its number. Keep it under 150 words.

SESSION

06

60 MIN · OOP

Introduction to Object-Oriented Programming

◆ Learning Objectives

- Explain why OOP was created and how it addresses the limitations of procedural programming
- Define the four pillars of OOP: Encapsulation, Inheritance, Polymorphism, Abstraction
- Map real-world wallet domain entities (User, Wallet, Transaction) to Java classes and objects
- Explain the difference between a class (blueprint) and an object (instance)
- Create a simple class with attributes and instantiate objects from it

01 Procedural vs Object-Oriented Programming

In procedural programming (like early C), code is a sequence of functions that operate on global data. As programs grow, this becomes unmanageable — changing one function breaks others, data is exposed everywhere, and there is no natural way to model real-world concepts. OOP solves this by bundling data and the functions that operate on it into self-contained units called objects.

Procedural Programming

Separate data (variables) and behavior (functions). Data is passed around and shared. Hard to scale, test, or maintain as complexity grows. Example: C, older PHP scripts.

Object-Oriented Programming

Data (fields) and behavior (methods) bundled into objects. Objects communicate by calling each other's methods. Scales naturally to large systems. Example: Java, Python, C#.

CONCEPT — The Four Pillars of OOP

Encapsulation: hide internal data, expose only what is needed. Inheritance: a child class reuses and extends a parent class. Polymorphism: the same method name behaves differently in different contexts. Abstraction: hide complex implementation details behind a simple interface. You will implement each pillar in detail in Sessions 07-11 — today you build the mental model.

02 Classes & Objects — Blueprint and Instance

A class is a blueprint — it defines what properties (fields) and behaviours (methods) an entity has, but does not hold actual data. An object is an instance created from that blueprint — it holds actual values. You can create many objects from one class, just as a bank creates millions of accounts from one Account schema.

CONCEPT — Class is the Schema, Object is the Record

Think of a class like a database table definition (CREATE TABLE users — it defines columns but has no rows

yet). An object is like one row in that table (INSERT INTO users...). A field in a class is like a column; a method is like a stored procedure that operates on that row's data. You can have one class and millions of objects, just like one table and millions of rows.

03 Modelling the Digital Wallet Domain in OOP

The Digital Wallet application we build throughout this module has three core entities: User (who owns the wallet), Wallet (the account holding the balance), and Transaction (each credit or debit event). Each maps naturally to a Java class. Their relationships mirror foreign key relationships in a database.

Real-World Entity	Java Class	Key Fields	Key Methods
Wallet User	User	name, email, mobile, pin	register(), login(), getProfile()
Digital Wallet	Wallet	balance, walletId, owner	addMoney(), deduct(), getBalance()
Each Payment	Transaction	amount, type, timestamp	getReceipt(), isFailed()

```
// Defining the User class (blueprint)
```

```
class User {
```

```
    String name;
```

```
    String email;
```

```
    double walletBalance;
```

```
    void displayInfo() {
```

```
        System.out.println(name + " | " + email + " | Rs." + walletBalance);
```

```
    }
```

```
}
```

Class declaration — no object yet

Field: stores the user's name

Field: stores email

Field: current balance

Method: defines what the object can do

04 Creating Objects & Accessing Members

You create an object using the new keyword followed by the class constructor. Each object gets its own copy of the class's instance fields. You access fields and methods using dot notation: `objectName.fieldName` or `objectName.methodName()`. Creating two User objects creates two independent sets of field values.

```
// Creating objects from the User blueprint
```

```
User user1 = new User();
```

user1 is a reference variable pointing to a new User object

```
user1.name = "Ankit Shah";  
user1.email = "ankit@example.com";  
user1.walletBalance = 5000.0;  
user1.displayInfo();
```

Set the name field on this specific object

Call the method on user1

```
User user2 = new User();  
user2.name = "Priya Patel";  
user2.walletBalance = 12000.0;  
user2.displayInfo();
```

A completely separate object — its own fields

Changing user2.name does not affect user1.name

Call same method — different data, different output

TRY THIS LIVE

Create a Wallet class with fields: walletId (String), ownerName (String), balance (double). Add a method showBalance() that prints all three fields. In main(), create two Wallet objects for two different users, assign values to their fields, and call showBalance() on each. Verify the objects are independent.

CASE STUDY — Flipkart: OOP Design at the Scale of India's Largest E-Commerce Platform

Context — Flipkart, headquartered in Bengaluru, handles millions of product listings, users, orders, and sellers — each a distinct entity with its own data and behaviours.

Challenge — Model a marketplace with millions of distinct entities — products, users, sellers, orders, carts, reviews — without creating an unmanageable tangle of global variables and functions.

What they did — Flipkart's backend uses Java with deep OOP design. Each entity (Product, User, Order, Cart) is a class. The separation of data and behaviour into objects allows teams to work on Product independently of Order without breaking each other's code.

Result — Hundreds of engineers work on Flipkart's codebase simultaneously. New features (like Quick Commerce) are added by creating new classes that extend existing ones — without rewriting the entire system.

Takeaway — Every time you model an entity as a class rather than a pile of variables, you are making your code maintainable, testable, and scalable — the way professional systems like Flipkart are built.

Source: Flipkart Engineering Blog, tech.flipkart.com (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 11-12: Introduction to OOP, Classes and Objects (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You understand why OOP exists and can create basic classes and objects. Next, deepen this with properly structured methods and member types.

Explore deeper — Java OOP Concepts — docs.oracle.com/javase/tutorial/java/concepts/

Try on your own — Add a Transaction class to your project with fields: txnId, amount, type (CREDIT/DEBIT), and a printReceipt() method. Create 3 Transaction objects and call printReceipt() on each.

Connect the dots — Session 07 goes deeper into methods — parameters, return types, and the difference between instance and static members.

Going further — Read about UML Class Diagrams — the standard way to visually design class structures before writing code, widely used in system design interviews.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as an OOP mentor. Explain the difference between a class and an object in simple Hinglish using the analogy of an Aadhaar card form. The blank form is the class (blueprint with fields: name, DOB, address). Each filled and submitted form is an object (actual data). Explain fields and methods in the same analogy. Keep it under 150 words.

SESSION

07

60 MIN · OOP

Classes, Objects & Methods

◆ Learning Objectives

- Design a class with well-structured instance fields and methods
- Distinguish between instance members (per-object) and static members (per-class)
- Define methods with parameters, return types, and meaningful names
- Call methods on objects and use return values correctly
- Build a Wallet class with addMoney() and checkBalance() methods

01 Designing a Well-Structured Class

A good Java class has a clear single responsibility — it models one thing and does it well. The Wallet class should only deal with wallet operations: adding funds, deducting, and showing balance. It should not also handle user authentication or file storage. This Single Responsibility Principle is the first rule of professional Java design.

CONCEPT — Instance vs Static Members

Instance fields and methods belong to each object separately — every Wallet object has its own balance field. Static fields and methods belong to the class itself — all objects share them. Use static for utility functions (Math.max(), Integer.parseInt()) and constants (Wallet.MAX_LIMIT). Use instance members for data that varies per object. Calling a static method: ClassName.method(). Calling an instance method: objectRef.method().

02 Instance Fields & Access

Instance fields are declared inside the class but outside any method. Each object created from the class gets its own independent copy of all instance fields. When you call new Wallet(), Java allocates memory for all the instance fields of that specific object on the heap.

```
class Wallet {
    // Instance fields — each wallet object has its own copy
    String walletId;           Unique ID per wallet
    String ownerName;         Owner's name
    double balance;           Current balance
    int transactionCount;      Number of transactions performed

    // Static field — shared by ALL Wallet objects
```

```
static double MAX_TRANSFER_LIMIT = 100000.0;
```

Same value for every wallet

```
static int totalWalletsCreated = 0;
```

Count across all objects

03 Defining Methods — Parameters & Return Types

A method defines what an object can do. The return type declares what value the method gives back — use void if it gives nothing back. Parameters are input values the caller must provide. Methods are like CRUD operations on the object's fields: they read or modify the object's state. Always name methods with a verb: addMoney(), checkBalance(), transferFunds().

```
// void method – performs an action, returns nothing
void addMoney(double amount) {
    if (amount <= 0) {
        System.out.println("Invalid amount");
        return;
    }
    balance += amount;
    transactionCount++;
    System.out.println("Added Rs." + amount + ". New balance: " + balance);
}

// Method with return type – gives a value back to caller
double checkBalance() {
    return balance;
}

// Method with parameter + return
boolean canTransfer(double amount) {
    return balance >= amount && amount <= MAX_TRANSFER_LIMIT;
}
```

Takes amount as input

Validate input — never trust raw data

Early exit from void method

Update instance field

Increment counter

Returns a double

Caller receives this value

Checks if transfer is possible

Returns true or false

04 Creating Objects & Calling Methods

Instantiate with new, then call methods using dot notation. The return value of a non-void method can be stored in a variable or used directly in a condition. Notice how each object's methods operate on that object's own fields — calling wallet1.addMoney() updates wallet1's balance, not wallet2's.

```

public class WalletDemo {
    public static void main(String[] args) {
        Wallet wallet1 = new Wallet();           Create first Wallet object
        wallet1.walletId = "WAL001";
        wallet1.ownerName = "Ankit Shah";
        wallet1.balance = 0;                     Start at zero

        wallet1.addMoney(5000);                  Calls addMoney on wallet1 — prints confirmation
        wallet1.addMoney(2000);                  Calls again — balance is now 7000

        double bal = wallet1.checkBalance();     Capture return value
        System.out.println("Balance: Rs." + bal);
        Prints 7000.0

        boolean ok = wallet1.canTransfer(3000);
        Check if transfer is possible
        System.out.println("Transfer allowed: " + ok);
        Prints true
    }
}

```

TRY THIS LIVE

Add a deductMoney(double amount) method to the Wallet class that: (1) checks if balance is sufficient; (2) if yes, deducts and prints the new balance; (3) if no, prints 'Insufficient balance'. In main(), add Rs. 5000, then try to deduct Rs. 3000 (should succeed), then try Rs. 4000 (should fail with insufficient balance message).

CONCEPT — Methods Map to CRUD Operations

In Java backend development, every class typically has methods corresponding to database CRUD: addMoney() -> INSERT a transaction record; checkBalance() -> SELECT the balance; deductMoney() -> UPDATE the balance; closeWallet() -> DELETE / deactivate the record. This is the bridge between OOP and the database layer — you will implement this explicitly when you learn JDBC.

Method Pattern	Return Type	Analogy
void addMoney(double amt)	void	INSERT — add data, confirm but return nothing
double checkBalance()	double	SELECT — retrieve and return a value
boolean canTransfer(double)	boolean	Validation query — return true/false

Method Pattern	Return Type	Analogy
<code>void deductMoney(double)</code>	<code>void</code>	UPDATE — modify state, confirm in console
<code>static int getWalletCount()</code>	<code>int</code>	Aggregate query — count across all objects

CASE STUDY — Razorpay: Methods as the API Contract

Context — Razorpay, Bengaluru's leading payment gateway, processes payments for over 8 million Indian businesses including many Gujarat-based SMEs using Razorpay's APIs.

Challenge — Allow merchants to integrate payment features (create order, capture payment, issue refund) without exposing Razorpay's internal implementation.

What they did — Razorpay exposes a clean set of methods (`createOrder()`, `capturePayment()`, `issueRefund()`) on its Java SDK classes. Each method has a clear input (parameters), a clear output (return type), and hides all the complexity of bank routing, settlement calculations, and fraud checks inside its implementation.

Result — Merchants integrate payments in hours rather than weeks. The clean method design means Razorpay can update its internal banking logic without breaking any merchant's code.

Takeaway — Every method you define is a promise to the caller: 'give me these inputs, I will perform this action and return this output.' Keeping that promise consistent — regardless of internal changes — is what professional API design means.

Source: Razorpay Developer Docs, razorpay.com/docs (2023)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 13-14: Methods, Instance and Static Members (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can build classes with fields and methods. Next, learn how to initialise objects cleanly using constructors.

Explore deeper — Java Methods Tutorial — docs.oracle.com/javase/tutorial/java/javaOO/methods.html

Try on your own — Add a `printStatement()` method to `Wallet` that loops over an internal `int[]` of the last 5 transaction amounts (hardcoded for now) and prints each with its index as 'Txn #N: Rs.X'.

Connect the dots — Session 08 introduces constructors — a special method that runs when an object is created, letting you set initial field values in one clean step.

Going further — Read about method signatures and why Java can identify methods by name + parameter list (not return type) — this sets the foundation for method overloading next session.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java mentor. Explain the difference between instance methods and static methods in simple Hinglish. Use the Wallet class example: checkBalance() needs to know whose wallet it is (instance), but a MAX_LIMIT rule applies to all wallets (static). Give a real example of when to use each. Keep it under 150 words.

SESSION

08

60 MIN · OOP

Constructors & Method Overloading

◆ Learning Objectives

- Explain the role of a constructor and how it differs from a regular method
- Write default, parameterised, and copy constructors for a class
- Use the this keyword to distinguish instance fields from constructor parameters
- Apply method overloading to define multiple versions of the same logical operation
- Connect constructor overloading to different forms of record insertion in a database

01 What is a Constructor?

A constructor is a special block of code that runs automatically the moment an object is created with new. Its job is to initialise the object's fields to valid starting values. Without a constructor, instance fields get language defaults (0, null, false) — which may cause NullPointerExceptions when methods are called on an uninitialised object. A constructor has the same name as the class and no return type.

CONCEPT — Constructor vs Method

A constructor: same name as the class, no return type (not even void), called automatically by new, cannot be called again later. A method: any name, has a return type (including void), called explicitly whenever needed. Java provides a hidden default constructor (no arguments, does nothing) only if you write zero constructors — once you write any constructor, the default disappears.

02 Default & Parameterised Constructors

A default constructor takes no arguments and sets safe initial values. A parameterised constructor takes arguments so the caller can supply specific values at creation time — cleaner than creating an object with null fields and then setting each one separately. Always validate constructor parameters before assigning them.

```
class User {
    String name;
    String email;
    String kycStatus;
    double walletBalance;

    // Default constructor – minimum safe state
    User() {
        this.name = "Unknown";
    }
}
```

Same name as class, no return type
this refers to the current object's field

```

        this.kycStatus = "PENDING";
        this.walletBalance = 0.0;
    }

// Parameterised constructor — caller provides values
    User(String name, String email) {
        this.name = name;
        this.email = email;
        this.kycStatus = "PENDING";
        this.walletBalance = 0.0;
    }

```

Parameters shadow instance fields — use this.
this.name = instance field; name = parameter

Always starts as pending

CONCEPT — The 'this' Keyword

When a constructor parameter has the same name as an instance field (both called name), Java gets confused about which 'name' you mean. The this keyword explicitly refers to the current object's instance field. this.name = name means: 'set this object's name field to the parameter named name.' You will also use this() to call one constructor from another (constructor chaining).

03 Copy Constructor

A copy constructor creates a new object as a duplicate of an existing one. It takes an object of the same class as its parameter and copies all field values. Useful for creating trial accounts from existing profiles, or duplicating a wallet configuration for testing without modifying the original.

```

// Copy constructor
    User(User other) {
        this.name = other.name;
        this.email = other.email;
        this.kycStatus = other.kycStatus;
    this.walletBalance = other.walletBalance;
    }

// Using all three constructors
    User u1 = new User();
    User u2 = new User("Ankit", "ankit@example.com");
    User u3 = new User(u2);

```

Takes another User object as parameter
Copy each field from the source object

Default: name=Unknown, kycStatus=PENDING

Parameterised: specific values

Copy: u3 is an independent copy of u2

04 Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists in the same class. Java distinguishes them by the number or type of parameters — not by return type. Overloading models the real-world idea that the same action can be performed in multiple ways with different inputs.

```
class Wallet {
    double balance;

    // Overloaded addMoney methods
    void addMoney(double amount) {           Add a specific amount
        balance += amount;
        System.out.println("Added Rs." + amount);
    }

    void addMoney(double amount, String note) {
        Add with a note/description
        balance += amount;
        System.out.println("Added Rs." + amount + " - " + note);
    }

    void addMoney(int amount) {             Accepts int — different parameter type
        balance += amount;                  Java auto-widens int to double for storage
        System.out.println("Added (int) Rs." + amount);
    }
}
```

TRY THIS LIVE

Build a User class with three constructors: (1) default — sets name to 'Guest' and balance to 0; (2) parameterised — takes name and email; (3) copy — clones another User. In main(), create one object with each constructor and print all fields. Then add an overloaded printInfo() method: one version prints just the name, another prints name + email + balance.

Constructor Overloading

Multiple constructors for the same class — fast registration vs full registration. Like different forms of INSERT: insert with minimal fields, or insert with full data.

Method Overloading

Multiple methods with the same name but different parameters. Like a stored procedure that accepts either an int ID or a String email to look up the same record.

PRO TIP — Constructor Chaining with this()

Instead of duplicating initialisation code, call one constructor from another using `this()`. Example: in the parameterised constructor, call `this()` first to run default initialisation, then override specific fields. This keeps your code DRY (Don't Repeat Yourself) — changes to the default logic apply everywhere automatically.

CASE STUDY — OYO Rooms: Multiple Registration Flows Using Constructor Overloading

Context — OYO, India's largest hospitality chain founded in Gurugram, manages over 157,000 properties and handles multiple types of user registrations: guests, hotel owners, and corporate accounts.

Challenge — Support three distinct registration flows — quick guest signup (phone only), full individual registration (name, email, phone, KYC), and corporate account setup (all fields + company data) — without maintaining three separate classes.

What they did — OYO's user management uses constructor overloading: one `User` class with constructors for each registration depth. The quick constructor takes just a phone number; the full constructor takes all personal details; the corporate constructor takes the full set plus company fields.

Result — One clean `User` class serves all three registration paths. The Java compiler automatically calls the right constructor based on which arguments the registration screen passes.

Takeaway — Constructor overloading is not just a language feature — it is the practical solution every multi-flow registration system uses to keep domain models clean and maintainable.

Source: OYO Engineering Blog, tech.oyorooms.com (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 15-16: Constructors and Method Overloading (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can initialise objects cleanly and write flexible methods. Next, learn how classes can share and extend each other's code.

Explore deeper — Java Constructors — docs.oracle.com/javase/tutorial/java/javaOO/constructors.html

Try on your own — Add a `User` class to your Digital Wallet project. Implement all three constructors and overloaded `printProfile()` methods. Create 3 users via different constructors, then print their profiles using the matching overloaded print method.

Connect the dots — Session 09 introduces Inheritance — child classes reuse parent class constructors and fields via the `super` keyword.

Going further — Look up the Builder design pattern — an alternative to constructor overloading for classes with many optional fields, used widely in modern Java (e.g., `AlertDialog.Builder` on Android).

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java instructor. Explain constructor overloading in simple Hinglish using the analogy of filling a bank account opening form. The quick form only needs a phone number (1-parameter constructor). The full form needs name, email, KYC documents (multi-parameter constructor). The corporate form needs all that plus a company registration number. One class, multiple ways to create an object. Keep it under 150 words.

SESSION

09

60 MIN · OOP

Inheritance & Reusability

◆ Learning Objectives

- Explain how inheritance promotes code reuse and models IS-A relationships
- Use the extends keyword to create single, multilevel, and hierarchical inheritance hierarchies
- Access parent class members using the super keyword
- Distinguish inherited members from overridden and new members in a child class
- Build a PremiumUser class that extends User and adds cashback-specific behaviour

01 What is Inheritance & Why Use It?

Inheritance allows a child class to automatically inherit all non-private fields and methods from a parent class — and then add or modify behaviour. Without inheritance, PremiumUser would duplicate all of User's code. With inheritance, PremiumUser simply extends User and only adds what is new. This is the DRY principle applied to class design.

CONCEPT — IS-A Relationship

Inheritance models an IS-A relationship: PremiumUser IS-A User. A Car IS-A Vehicle. A SavingsAccount IS-A BankAccount. The IS-A test: can you say 'every [child] is a [parent]'? If yes, inheritance is appropriate. If the relationship is HAS-A (a Wallet HAS-A Transaction list), use composition instead — store an object as a field, not as a parent class.

02 extends — Single & Multilevel Inheritance

Java uses the extends keyword to declare inheritance. Java supports single inheritance only for classes — a child can have exactly one direct parent. Multilevel inheritance chains three or more levels: GrandParent -> Parent -> Child. Every class in Java implicitly extends Object — the root of all Java class hierarchies.

```
// Parent class
class User {
    String name;
    String email;
    double walletBalance;

    User(String name, String email) {
        this.name = name;
        this.email = email;
        this.walletBalance = 0;
    }
}
```

```

    }

    void login() {
System.out.println(name + " logged in.");
    }
}

// Child class – inherits everything non-private from User
class PremiumUser extends User {
    double cashbackRate;           PremiumUser IS-A User
    String tierName;                Extra field — only PremiumUser has this
                                   e.g., Gold, Platinum

    PremiumUser(String name, String email, double cashbackRate) {
        super(name, email);        MUST call parent constructor first using super()
        this.cashbackRate = cashbackRate;    Then initialise child-specific fields
        this.tierName = cashbackRate >= 5.0 ? "Platinum" : "Gold";
    }

    void applyCashback(double txnAmount) {    New method — only in PremiumUser
        double cashback = txnAmount * (cashbackRate / 100);
        walletBalance += cashback;           walletBalance inherited from User
        System.out.println("Cashback: Rs." + cashback + " applied.");
    }
}

```

CONCEPT — super Keyword

super refers to the parent class. super() in the first line of a child constructor calls the parent constructor — required if the parent has no default constructor. super.methodName() calls the parent's version of a method — useful when the child overrides a method but still wants to run the parent's logic first. If you do not call super() explicitly, Java tries to call the parent's no-arg constructor automatically.

03 Hierarchical Inheritance

Hierarchical inheritance means multiple child classes extend the same parent. In a digital payments system, you might have one base User class with distinct child classes: PremiumUser (cashback features), MerchantUser (business dashboard features), and SupportAgent (ticketing features). All inherit User's name, email, and login() — each adds its own domain-specific behaviour.

Inheritance Type	Structure	Use Case
Single	User -> PremiumUser	One parent, one child — most common

Inheritance Type	Structure	Use Case
Multilevel	Account -> WalletAccount -> PremiumWallet	Grandparent chain — use sparingly
Hierarchical	User -> PremiumUser, MerchantUser, SupportAgent	One parent, many children — models user roles
Multiple (NOT in Java)	A, B -> C	Java blocks this for classes — use interfaces instead

TRY THIS LIVE

Extend your User class with two child classes: (1) PremiumUser with a cashbackRate field and applyCashback(double txnAmount) method; (2) MerchantUser with a businessName field and showBusinessInfo() method. In main(), create one object of each, call their parent method login() (inherited — no need to rewrite), and then call their own specific methods.

PRO TIP — When NOT to Use Inheritance

Do not use inheritance just to reuse a few methods. If the IS-A relationship does not hold (a Wallet is NOT a User), use composition instead: give the User class a Wallet field. Overusing inheritance creates fragile class hierarchies where a change to the parent breaks all children unexpectedly. Modern Java design favours composition over inheritance for most reuse scenarios.

CASE STUDY — Jio: Inheritance Modelling Diverse Service Plans

Context — Reliance Jio, headquartered in Mumbai, manages hundreds of millions of subscribers across prepaid, postpaid, JioFiber, and JioBusiness plans.

Challenge — Represent millions of subscribers on vastly different plans — all sharing common attributes (phone number, billing address, payment method) but with plan-specific features (data quota, streaming access, device limits).

What they did — Jio's subscriber management system uses a base Subscriber class with common fields and billing methods. Each plan type (PrepaidSubscriber, PostpaidSubscriber, FiberSubscriber) extends Subscriber and adds plan-specific fields and methods. All share the base login, payment processing, and notification logic.

Result — New plan types are added by creating a new child class — without touching any existing code. The hierarchy supports hundreds of plan variants without code duplication.

Takeaway — Inheritance is how enterprise systems add new product types without rewriting existing logic. The extends keyword is the mechanism that makes large-scale software maintainable and extensible.

Source: Jio Platform Engineering overview, publicly discussed at India mobile conferences (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 17-18: Inheritance, super and this (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can build class hierarchies that reuse and extend behaviour. Next, learn how the same method name can behave differently across the hierarchy.

Explore deeper — Java Inheritance — docs.oracle.com/javase/tutorial/java/landl/subclasses.html

Try on your own — Add a third level: VIPUser extends PremiumUser — add a personalRelationshipManager (String) field and a contactRM() method. Create a VIPUser and call login() (from User), applyCashback() (from PremiumUser), and contactRM() (its own). Verify all three levels work.

Connect the dots — Session 10 covers polymorphism — how Java decides which version of an overridden method to call when you have a parent reference pointing to a child object.

Going further — Look up Java's Object class methods — toString(), equals(), and hashCode() — which every class inherits automatically. Overriding these is standard practice in professional Java code.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java mentor. Explain inheritance in simple Hinglish using the analogy of a government employee hierarchy: a basic Employee class (attendance, salary), then Manager extends Employee (team size, appraisal method), then Director extends Manager (budget approval). Explain what gets inherited, what gets added, and what super does. Keep it under 150 words.

SESSION

10

60 MIN · OOP

Polymorphism

◆ Learning Objectives

- Distinguish compile-time polymorphism (method overloading) from runtime polymorphism (method overriding)
- Use `@Override` to override a parent method in a child class correctly
- Explain dynamic dispatch and how Java selects the method version at runtime
- Hold child objects in parent-type references and call overridden methods via a loop
- Model multiple wallet top-up channels using a polymorphic class hierarchy

01 What is Polymorphism?

Polymorphism means 'many forms' — the same action behaves differently depending on the object performing it. In a digital wallet, `processTopUp()` works differently for a card, a UPI transfer, and a crypto conversion, yet all three share the same method name. Polymorphism lets your code work uniformly with any payment type without needing if-else blocks to check the type at every step.

Compile-Time (Static) Polymorphism

Resolved at compile time via method overloading — same method name, different parameter lists. The compiler picks the right version by matching argument types. Also called early binding.

Runtime (Dynamic) Polymorphism

Resolved at runtime via method overriding — child redefines a parent method. Java looks at the actual object type, not the reference type. Also called late binding or dynamic dispatch.

02 Method Overriding & `@Override`

A child overrides a parent method by writing a method with the exact same name, return type, and parameter list. The `@Override` annotation asks the compiler to verify the override is valid — a typo in the method name would otherwise silently create a brand-new method rather than an override. Always use `@Override`; it is the only protection against this class of silent bug.

```
class TopUp {                                     Parent — generic top-up
    void processTopUp(double amount) {
        System.out.println("Processing generic top-up: Rs." + amount);
    }
}
class CardTopUp extends TopUp {
    @Override                                     Compiler verifies this actually overrides something
    void processTopUp(double amount) {           Same signature
        System.out.println("Card charged: Rs." + amount + " | 2% convenience fee");
    }
}
```

```

    }
}
class UPITopUp extends TopUp {
    @Override
    void processTopUp(double amount) {
        System.out.println("UPI transfer: Rs." + amount + " | Instant, zero fee");
    }
}
class CryptoTopUp extends TopUp {
    @Override
    void processTopUp(double amount) {
        System.out.println("Crypto converted: Rs." + amount + " at live BTC rate");
    }
}

```

03 Dynamic Dispatch — Runtime Method Selection

The power of runtime polymorphism is writing code that uses a parent-type reference but automatically calls the child's overridden method. Java resolves which version to call at runtime by inspecting the actual object type, not the declared type of the reference variable. This lets you add new child classes without changing any existing calling code.

```

TopUp[] channels = {new CardTopUp(), new UPITopUp(), new CryptoTopUp()};
    Parent-type array holding child objects
for (TopUp t : channels) {                                Iterate over all channels
    t.processTopUp(1000);                                    Java picks the correct child method at runtime
}
// Output:
// Card charged: Rs.1000.0 | 2% convenience fee
// UPI transfer: Rs.1000.0 | Instant, zero fee
// Crypto converted: Rs.1000.0 at live BTC rate

```

CONCEPT — Open/Closed Principle via Polymorphism

Without polymorphism, adding a new top-up channel means editing the main if-else block — touching existing, tested code. With polymorphism, you create a new child class and the loop above works immediately. This is the Open/Closed Principle: software should be open for extension but closed for modification. Polymorphism is Java's primary mechanism for achieving it.

TRY THIS LIVE

Create a base class Notification with a method send(String message). Write three children: SMSNotification, EmailNotification, WhatsAppNotification — each overrides send() with its own output. Build a Notification[] array with one of each, loop over it calling send(). Add a PushNotification child and verify the loop requires zero changes.

PRO TIP — instanceof for Safe Downcasting

When you have a parent reference and need to call a child-specific method, downcast: CardTopUp card = (CardTopUp) t. But if t actually holds a UPITopUp, this throws ClassCastException. Always check first: if (t instanceof CardTopUp card) { card.cardSpecificMethod(); }. Java 16+ pattern matching combines the check and cast in one line.

DESIGN PATTERNS USING POLYMORPHISM

70%

Approximately 70% of the classic GoF patterns (Strategy, Template Method, Factory, Observer) rely on method overriding — Head First Design Patterns, O'Reilly (2020)

CASE STUDY — Zomato: Polymorphism Across Delivery Types

Context — Zomato, headquartered in Gurugram, serves 500+ Indian cities with multiple delivery models: partner restaurant, Zomato Instant, Intercity Legends, and scheduled delivery.

Challenge — Calculate ETA and delivery fee uniformly across all delivery types without a giant if-else in the assignment engine — new delivery types are added frequently.

What they did — A base Delivery class defines calculateETA() and computeFee(). Each delivery type (BikeDelivery, InstantPickup, IntercityParcel) extends Delivery and overrides both methods. The core dispatch engine iterates over Delivery[] and calls overridden methods — oblivious to which specific type it holds.

Result — Adding a new delivery type (e.g., drone delivery) requires only a new child class with its own ETA and fee logic — zero changes to the dispatch engine that routes millions of orders daily.

Takeaway — Dynamic dispatch is what makes large systems extensible without becoming fragile. Every new payment mode, delivery type, or notification channel in any major Indian app is a new child class — not a new if-else block.

Source: Zomato Engineering Blog, engineering.zomato.com (2023)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 19-20: Polymorphism and Dynamic Dispatch (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can design extensible class hierarchies using polymorphism. Next, protect data with access modifiers and define method contracts using abstraction.

Explore deeper — Java Polymorphism — docs.oracle.com/javase/tutorial/java/landl/polymorphism.html

Try on your own — Add a `getFeeBreakdown()` method to each `TopUp` child that returns a `String` describing the fees for Rs. 5000. Store all three in a `TopUp[]` and print fee breakdowns in a single loop.

Connect the dots — Session 11 covers Encapsulation and Abstraction — hiding data behind private fields and defining method contracts via abstract classes and interfaces.

Going further — Research the Strategy design pattern — it uses polymorphism to swap algorithms (sort strategies, payment strategies) at runtime without changing the calling code, making it the architectural embodiment of dynamic dispatch.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java instructor. Explain runtime polymorphism and dynamic dispatch in simple Hinglish. Use the analogy of a bank counter: you go to the 'Service' counter (parent reference), but the actual person (child object) might be a trainee, senior teller, or manager — each handles your transaction request (same method call) differently. Keep it under 150 words.

SESSION

11

60 MIN · OOP

Encapsulation & Abstraction

◆ Learning Objectives

- Apply the four access modifiers (public, protected, default, private) correctly
- Implement getters and setters to expose private fields with validation
- Explain when to use an abstract class versus an interface
- Design an abstract Payment class with processPayment() as an abstract method
- Connect abstraction to API contracts between service and database layers

01 Encapsulation — Protecting Your Data

Encapsulation bundles data (fields) and the methods that act on it inside a class, restricting direct external access. Without encapsulation, any code can write `wallet.balance = -99999` — a catastrophic security hole in any fintech application. Encapsulation forces all changes through validated methods that can check rules, log operations, and reject bad input before the field is touched.

Modifier	Same Class	Same Package	Subclass (any pkg)	Everywhere
private	[ok]	[x]	[x]	[x]
default (none)	[ok]	[ok]	[x]	[x]
protected	[ok]	[ok]	[ok]	[x]
public	[ok]	[ok]	[ok]	[ok]

CONCEPT — Standard Encapsulation Pattern

Declare all instance fields private. Provide a public getter (`getXxx()`) for read access and a public setter (`setXxx()`) for write access with validation. This is so universally used that every Java IDE has a 'Generate Getters and Setters' shortcut. Following this pattern means your fields can never be corrupted by code outside the class — the class itself is the sole guardian of its data.

02 Getters, Setters & Validation

```
class Wallet {
    private String walletId;
    private double balance;
```

private — inaccessible from outside
Cannot be set directly from any other class

```

    private String ownerEmail;

    public double getBalance() { return balance; }
    Getter — read-only external access
    public String getWalletId() { return walletId; }

    public void setOwnerEmail(String email) {      Setter with business rule
    if (email != null && email.contains("@")) {
        Validate before accepting
        this.ownerEmail = email;
    } else {
        throw new IllegalArgumentException("Invalid email: " + email);
        Reject bad data loudly
    }
    }

    public void credit(double amount) {            Controlled balance update
    if (amount > 0) { balance += amount; }
        Validation inside — no negative credits
    else { throw new IllegalArgumentException("Credit must be positive"); }
    }
    }

    // Calling code
    Wallet w = new Wallet();
    // w.balance = 5000; <- COMPILE ERROR — private field
    w.credit(5000);                                [ok] Goes through validation
    System.out.println(w.getBalance());             [ok] Read via getter

```

TRY THIS LIVE

Make all fields in your User class private. Add getters for name, email, and walletBalance. Add setters for email (reject if missing '@') and walletBalance (reject if negative). In main(), try setting a valid and an invalid email — observe the exception for the invalid one. Then try to access a private field directly — observe the compile error.

03 Abstraction — Abstract Classes

Abstraction hides implementation complexity and exposes only what is necessary. In Java, abstract classes mix abstract methods (no body — subclasses must implement) with concrete methods (fully implemented — inherited for free). Mark the class abstract; instantiating it directly is a compile error — only concrete subclasses can be instantiated.

```
abstract class Payment {
    protected double amount;
    protected String txnId;

    public abstract void processPayment();

    public void printReceipt() {
        System.out.println("TxnId: " + txnId + " | Amount: Rs." + amount);
    }
}

class UPIPayment extends Payment {
    private String upiId;
    UPIPayment(double amount, String upiId) {
        this.amount = amount; this.upiId = upiId;
        this.txnId = "UPI" + System.currentTimeMillis();
    }
    @Override
    public void processPayment() {
        System.out.println("Debiting Rs." + amount + " via UPI: " + upiId);
    }
}
```

Cannot instantiate Payment directly
Shared field — accessible to subclasses

No body — contract every subclass must fulfil

Concrete — inherited by all subclasses for free

Must implement — abstract contract fulfilled

04 Interfaces — Pure Contracts

An interface defines what a class can do, with no implementation details. Before Java 8, all interface methods were abstract. Since Java 8, interfaces can have default methods (with a body, inherited) and static methods. A class can implement multiple interfaces — Java's solution to multiple inheritance. Use interfaces to define capability contracts: Payable, Printable, Serializable, Auditable.

```
interface Payable {
    void pay(double amount);
    String getPaymentSummary();
    default void logPayment() {
        System.out.println("Payment logged at " + java.time.LocalDateTime.now());
    }
}

class CardPayment implements Payable {
    private String cardNumber;
    CardPayment(String cardNumber) { this.cardNumber = cardNumber; }
    @Override
    public void pay(double amount) {
```

Pure contract — what a payment object must do
Implicitly public abstract

Default method — optional to override

Must implement all abstract methods

```
System.out.println("Card *" + cardNumber.substring(cardNumber.length()-4) + " charged:
Rs." + amount);
}
@Override
public String getPaymentSummary() {
return "Card payment via *" + cardNumber.substring(cardNumber.length()-4);
}
}
```

Abstract Class

Use when child classes share common code and state. Can have fields, constructors, and concrete methods. A child class extends only ONE abstract class.

Interface

Pure contract — defines what a class can do, not what it is. No instance fields. A class implements MULTIPLE interfaces. Use for capability contracts: Payable, Printable, Closeable.

CASE STUDY — HDFC Bank: Encapsulation Protecting Account Balances

Context — HDFC Bank, India's largest private bank, manages over 75 million customer accounts through NetBanking, mobile app, and branch channels.

Challenge — Ensure account balances can only be modified through validated, audited transactions — no code path should allow arbitrary balance changes that bypass business rules.

What they did — Account balances are private fields. All credits and debits pass through public methods (credit(), debit(), transferFunds()) that validate daily limits, KYC status, account freeze flags, and write audit trail entries before updating the balance. Direct field access from outside the class is structurally impossible.

Result — No cases of unauthorised balance manipulation through software vulnerabilities in the core banking layer — encapsulation makes it architecturally impossible to bypass business rules.

Takeaway — private fields with validated setters are not restrictive — they are the first line of defence that makes financial software trustworthy at the code level, regardless of who writes future code against the class.

Source: HDFC Bank Technology Architecture publicly discussed at Nasscom ILF (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 21-22: Encapsulation, Abstract Classes and Interfaces (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

All four OOP pillars are complete. Next, learn how Java stores, processes, and formats text — the foundation of all user-facing output.

Explore deeper — Java Interfaces Tutorial — docs.oracle.com/javase/tutorial/java/landl/createinterface.html

Try on your own — Create a Payable interface with `pay(double amount)` and `getPaymentSummary()`. Implement it in `CardPayment` and `WalletPayment`. Create both objects, add them to a `Payable[]` array, and call both methods on each. Add a new `NetBankingPayment` class and verify the array loop requires no changes.

Connect the dots — Session 12 covers String Handling — essential for formatting names, masking wallet IDs, building transaction descriptions, and processing user input.

Going further — Research `@FunctionalInterface` — an interface with exactly one abstract method that enables Java 8 lambda expressions, which you saw briefly in the sorting Comparator. Lambdas are the functional-programming layer built on top of interfaces.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java instructor. Explain encapsulation in simple Hinglish using the ATM analogy: you cannot touch the cash inside the ATM directly (private field). You must use the keypad and screen (public methods). The ATM validates your PIN and daily limit before dispensing (setter with business rules). Explain getters and setters in the same analogy. Keep it under 150 words.

SESSION

12

60 MIN · CORE JAVA

String Handling

◆ Learning Objectives

- Explain why String is immutable in Java and how the String Pool works
- Use common String methods: length(), substring(), equals(), contains(), split(), replace(), trim()
- Compare String, StringBuilder, and StringBuffer and choose the right one
- Format output using String.format() and System.out.printf()
- Build a wallet ID masking utility: WAL1234 -> ***1234

01 String is Immutable — And Why That Matters

In Java, String is a class, not a primitive. Every String is immutable: once created, its characters cannot be changed. When you write `id = id + "-ACTIVE"`, Java does not modify the original String — it creates a new String object, makes the variable point to it, and leaves the old one for garbage collection. This matters in loops: concatenating in a loop creates hundreds of throw-away objects, degrading performance.

CONCEPT — String Pool

When you write `String s = "Hello"`, Java stores "Hello" in a special String Pool in the heap and reuses it for all identical literals. Two variables pointing to the same literal share the same pool object — which is why `==` can return true between two literals. But `new String("Hello")` always creates a new heap object outside the pool. Conclusion: always compare String content with `.equals()`, never with `==`.

02 Essential String Methods

<code>String walletId = "WAL-AHM-001234";</code>	14 characters
<code>String name = " Ankit Shah ";</code>	Leading and trailing spaces
<code>walletId.length()</code>	-> 14
<code>walletId.charAt(4)</code>	-> 'A' (index 4)
<code>walletId.substring(4)</code>	-> 'AHM-001234' (from index 4 to end)
<code>walletId.substring(0, 3)</code>	-> 'WAL' (end index is exclusive)
<code>walletId.toLowerCase()</code>	-> 'wal-ahm-001234'
<code>walletId.toUpperCase()</code>	-> 'WAL-AHM-001234'
<code>walletId.contains("AHM")</code>	-> true
<code>walletId.replace("AHM", "SRT")</code>	-> 'WAL-SRT-001234' (new String — original unchanged)
<code>walletId.startsWith("WAL")</code>	-> true
<code>walletId.split("-")</code>	-> <code>String[] {"WAL", "AHM", "001234"}</code>

<code>name.trim()</code>	-> 'Ankit Shah' (removes edge spaces)
<code>walletId.equals("WAL-AHM-001234")</code>	-> true (use this, never ==)
<code>walletId.isEmpty()</code>	-> false (length > 0)
<code>String.valueOf(5000)</code>	-> '5000' (int to String conversion)
<code>Integer.parseInt("500")</code>	-> 500 (String to int)

03 String vs StringBuilder vs StringBuffer

Building a String by appending in a loop with + creates a new object every iteration — 1000 appends = ~1000 garbage objects. StringBuilder maintains a single mutable buffer and appends in-place — orders of magnitude faster for loop-based string construction. StringBuffer is thread-safe (synchronised) but slower — use only when multiple threads share the same buffer, which is rare.

String — Immutable

Each + creates a new object. Thread-safe by default. Use for fixed values, method parameters, and short concatenations (2–3 parts).

StringBuilder — Mutable

append() modifies in-place. No new objects per append. Use whenever building strings in a loop or assembling multi-part messages. Call toString() once at the end.

```
// BAD — String concatenation in loop
String report = "";
for (int i = 0; i < 100; i++) {
    report += "Txn-" + i + "\n";
}
```

New String object each iteration — 100 garbage objects

```
// GOOD — StringBuilder
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100; i++) {
    sb.append("Txn-").append(i).append("\n");
}
String report = sb.toString();
```

One mutable buffer

Chained append — no new objects

Single conversion to immutable String at the end

```
// Other useful StringBuilder methods
sb.insert(0, "HEADER\n");
sb.reverse();
sb.delete(0, 7);
```

Insert at position 0

Reverses content in-place

Delete chars from index 0 to 6

04 Formatting & Wallet ID Masking

`String.format()` creates a formatted String using placeholders: `%s` (String), `%d` (integer), `%f` (float), `%.2f` (float to 2 decimal places). `printf()` does the same but prints directly. Masking — hiding sensitive characters in an ID or phone number — is a real security requirement in every fintech app; it uses `length()` and `substring()` together.

```
// Wallet ID masking: WAL1234 -> ***1234
String walletId = "WAL1234";
String masked = "****" + walletId.substring(walletId.length() - 4);
    Keep last 4 chars
System.out.println(masked);                Prints: ***1234

// Phone masking: +919876543210 -> +91*****3210
String phone = "+919876543210";
String maskedPhone = phone.substring(0,3) + "*****" +
phone.substring(phone.length()-4);
System.out.println(maskedPhone);           Prints: +91*****3210

// Formatted statement line
String line = String.format("%-15s %-8s %10.2f", "TXN-001", "CREDIT", 5000.0);
System.out.println(line);                 TXN-001    CREDIT    5000.00

System.out.printf("Balance: Rs. %, .2f%n", 125000.50);
    With thousand-separator comma
```

TRY THIS LIVE

Write a `maskData(String input)` method: replace all characters except the last 4 with '*'. Test with 'WAL-001234' -> '*****1234' and '+919898001234' -> '*****1234'. Then build a 5-row mini-statement using `StringBuilder`: each row has a formatted `txnId` (`%-10s`), type (`%-8s`), and amount (`%10.2f`). Print the full statement in one `println()`.

PRO TIP — `toCharArray()` and `charAt()` for Custom Masking

For fine-grained masking, convert the String to a char array with `toCharArray()`, modify the array in a loop, and rebuild with new `String(charArray)`. This is cleaner than multiple `substring()` calls when the masking rule is complex — for example, revealing every 3rd character of a 16-digit card number while hiding the rest.

CASE STUDY — Ola: String Processing for Ride Receipt Generation

Context — Ola, the Bengaluru-based ride-hailing platform, generates millions of digital receipts daily for rides across 250+ Indian cities.

Challenge — Generate formatted, accurate receipts with masked phone numbers, formatted fare amounts, and properly aligned text — at thousands per second during peak hours.

What they did — Ola's receipt service uses `StringBuilder` to assemble multi-line receipts without creating thousands of intermediate `String` objects. `String.format()` formats rupee amounts to two decimal places. `substring()` masks driver and passenger phone numbers (last 4 digits only) in compliance with privacy regulations.

Result — Receipts are generated in milliseconds at scale, with correctly masked phone numbers, formatted INR amounts, and clean columnar alignment — all using `String` operations you practised today.

Takeaway — `String` manipulation is not a beginner exercise — masking sensitive data, formatting currency, and building dynamic text at production scale is a real engineering requirement in every customer-facing product.

Source: *Ola Engineering Blog*, tech.olacabs.com (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lecture 23: String Handling, `StringBuilder`, `StringBuffer` (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can manipulate and format text professionally. Next, learn how to handle runtime errors gracefully so your program never crashes on invalid input.

Explore deeper — Java String API —

docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html

Try on your own — Build a `WalletStatement` class: use `StringBuilder` to produce a formatted 6-row statement (header + 5 transactions). Each row: masked txnid, type, amount formatted to 2 decimal places. Print the full statement with one `println()`.

Connect the dots — Session 13 introduces Exception Handling — using try-catch-finally so errors like invalid amounts, null wallet IDs, or insufficient balance produce clear messages rather than crashes.

Going further — Look up Java's regex support (`Pattern` and `Matcher` classes) — used for validating UPI IDs (format: `handle@bank`), phone numbers, and email addresses in real registration flows.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java tutor. Explain the difference between `String`, `StringBuilder`, and `StringBuffer` in simple Hinglish using a whiteboard analogy: `String` = write on paper and throw away for every change; `StringBuilder` = reusable whiteboard (erase and update in place, fast); `StringBuffer` = whiteboard with a door lock (safe for two people editing at once, but slower). Keep it under 150 words.

SESSION

13

60 MIN · CORE JAVA

Exception Handling – Basics

◆ Learning Objectives

- Define what an exception is and explain why unhandled exceptions crash programs
- Distinguish Exceptions (recoverable) from Errors (JVM-level, unrecoverable)
- Write try-catch-finally blocks to handle exceptions without crashing
- Differentiate checked exceptions (compiler-enforced) from unchecked (runtime)
- Use throw and throws to signal and declare exceptions from methods

01 What is an Exception?

An exception is an unexpected event that disrupts the normal flow of a program at runtime — dividing by zero, accessing a null reference, parsing 'abc' as an integer, or withdrawing more than the available balance. Without exception handling, the JVM prints a stack trace and terminates the program immediately — unacceptable in any production application, particularly a financial one processing crores of rupees.

CONCEPT — Exception vs Error

Exception: a recoverable problem your program can anticipate and handle — `NullPointerException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`, `IOException`. Error: a severe JVM-level condition programs cannot normally recover from — `OutOfMemoryError`, `StackOverflowError`. The rule: catch Exceptions. Never catch Errors unless you are writing JVM infrastructure. Both extend `Throwable` — the root of Java's error hierarchy.

02 try-catch-finally

The try block holds code that might throw. The catch block handles one specific exception type and runs only if that exception occurs. The finally block always runs — whether the try succeeded or threw — making it the correct place to close resources, print status, or roll back partial operations.

```
double balance = 2000; double withdrawal = 3000;
```

```
try {
    if (withdrawal > balance) {
        throw new Exception("Insufficient balance");
    }
    balance -= withdrawal;
```

Code that might fail

Manually throw — stops execution here

Skipped if exception thrown above

```

System.out.println("New balance: Rs." + balance);
} catch (Exception e) {
    // Runs only when exception was thrown
    System.out.println("Transaction failed: " + e.getMessage());
    // getMessage() returns 'Insufficient balance'
} finally {
    // ALWAYS runs — success or failure
    System.out.println("Attempt recorded in audit log.");
    // Release resources, log completion here
}

```

03 Checked vs Unchecked Exceptions

Type	Common Examples	Compiler Forces Handling?	Typical Cause
Checked	IOException, SQLException, FileNotFoundException	Yes	External failures: file, network, DB
Unchecked (RuntimeException)	NullPointerException, NumberFormatException, ArrayIndexOutOfBoundsException	No — optional	Programming mistakes, invalid input
Error	OutOfMemoryError, StackOverflowError	No — do not catch	JVM-level resource exhaustion

CONCEPT — Checked Exceptions Are Compile-Time Contracts

When a method declares throws IOException, the compiler forces every caller to either wrap the call in try-catch or re-declare throws IOException in its own signature. This propagates the responsibility up the call stack until someone handles it. Unchecked exceptions have no such requirement — they bubble up and crash the program if unhandled. The distinction is: external failures (checked, unpredictable) vs programming errors (unchecked, preventable with validation).

04 throw & throws

```

// 'throws' on method signature — declares checked exceptions
void withdraw(double amount) throws Exception {
    // Caller must handle this
    if (amount <= 0) {
        throw new IllegalArgumentException("Amount must be positive");
    }
    // Unchecked — no throws needed
}

```

```

    }
    if (amount > balance) {
throw new Exception("Insufficient funds: have Rs." + balance);
    Checked — must be in throws
    }
    balance -= amount;
    Only runs if no exception thrown above
    System.out.println("Withdrawn. Remaining: Rs." + balance);
}

// Calling code must handle the checked exception
try {
    wallet.withdraw(5000);
} catch (Exception e) {
    System.out.println("Could not withdraw: " + e.getMessage());
}

```

TRY THIS LIVE

Add a `withdraw(double amount)` method to `Wallet`. Use `throw` to raise `IllegalArgumentException` for `amount <= 0`, and `throw new Exception` for `amount > balance`. In `main()`, write three separate try-catch blocks: (1) valid withdrawal, (2) negative amount, (3) insufficient funds. Print the exact error message from each catch block and a 'finally: attempt logged' message from each finally block.

WARNING — Never Swallow Exceptions

`catch (Exception e) { }` — an empty catch block — is one of the most dangerous patterns in Java. The exception is completely hidden: no error message, no log, no way to diagnose the problem later. Always at minimum print `e.getMessage()` or log the exception. In production, an empty catch block turns a 5-minute debug into a 5-hour mystery.

CASE STUDY — MakeMyTrip: Exception Handling Across a Live Booking Flow

Context — MakeMyTrip, India's largest online travel platform, handles flight, hotel, and holiday bookings with real-time inventory and payments.

Challenge — Handle dozens of live failure modes gracefully: seat sold between search and checkout, payment gateway timeout, hotel API error, cancellation policy mismatch — without crashing the booking flow or showing a generic error.

What they did — Every external API call is wrapped in try-catch with specific exception types per failure mode. A `PaymentGatewayTimeoutException` triggers a retry. A `SeatUnavailableException` redirects to alternate options. The finally block always releases the temporary seat hold — whether the booking succeeded or not — preventing ghost reservations.

Result — MakeMyTrip maintains 99.9% uptime on the booking flow even during Diwali peak, routing each

failure to a specific recovery path instead of a crash screen.

Takeaway — Exception handling is not about hiding errors — it is about routing them. Every catch block is a controlled decision: 'when this specific thing goes wrong, take this specific alternative action.' That is what separates robust software from fragile software.

Source: MakeMyTrip Engineering Blog, tech.makemytrip.com (2023)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lecture 24: Exception Handling — try, catch, finally, throw, throws (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can handle exceptions gracefully. Next, handle multiple exception types and build your own custom exception classes for domain-specific failures.

Explore deeper — Java Exceptions Tutorial — docs.oracle.com/javase/tutorial/essential/exceptions/

Try on your own — Wrap your wallet application's Scanner input in try-catch. Handle NumberFormatException when the user types letters instead of an amount. Use a do-while loop to keep asking until valid input is received, then proceed with the transaction.

Connect the dots — Session 14 covers multiple catch blocks, custom exception hierarchies, and try-with-resources — the patterns used in production-grade exception handling.

Going further — Research Java 7's try-with-resources syntax: resources declared in try() are automatically closed when the block exits, eliminating the need for finally to close files or DB connections.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java instructor. Explain try-catch-finally in simple Hinglish using a train booking analogy: try = attempt the booking; catch = show a specific helpful message if the seat is taken or payment fails; finally = always release the temporary hold on the seat (so it becomes available for others), whether booking succeeded or not. Keep it under 150 words.

SESSION

14

60 MIN · CORE JAVA

Exception Handling – Advanced

◆ Learning Objectives

- Use multiple catch blocks to handle different exception types from one try block
- Create a custom `InsufficientFundsException` with a meaningful message and error code
- Apply try-with-resources to guarantee automatic resource closure
- Chain exceptions to preserve the original stack trace when re-throwing
- Design a complete multi-step transaction flow with layered exception handling

01 Multiple catch Blocks

A single try block may throw different exceptions depending on what fails. Multiple catch blocks handle each type separately — so a `NumberFormatException` produces 'Please enter a valid number' while an `InsufficientFundsException` produces 'Top up required'. Java checks catch blocks top to bottom and executes the first match only. Always order from most specific (child) to most general (parent).

```
try {
    String input = scanner.nextLine();           May be null or non-numeric
    double amount = Double.parseDouble(input);
    Throws NumberFormatException if non-numeric
    wallet.withdraw(amount);                     Throws InsufficientFundsException if low balance
    int[] log = new int[5]; log[10] = 1;         Throws ArrayIndexOutOfBoundsException
} catch (NumberFormatException e) {             Most specific — catch first
    System.out.println("Invalid input: " + e.getMessage());
} catch (InsufficientFundsException e) {        Custom domain exception
    System.out.println("Rejected [" + e.getErrorCode() + "]: " + e.getMessage());
    System.out.println("Top up at least: Rs." + e.getShortfall());
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Log error: " + e.getMessage());
} catch (Exception e) {                         Generic LAST — safety net
    System.out.println("Unexpected: " + e.getMessage());
} finally {
    System.out.println("Transaction attempt complete.");
    Always runs
}
```

CONCEPT — Catch Order: Child Before Parent

If you put `catch(Exception e)` before `catch(NumberFormatException e)`, the general block catches everything and the specific block is unreachable — the compiler flags this as a compile error. Always order: most specific

(deepest child in the hierarchy) first, most general (Exception or Throwable) last. The catch block is executed only once — after the first match, the rest are skipped.

02 Creating Custom Exceptions

Custom exceptions let you name domain failures explicitly and carry domain-specific data. `InsufficientFundsException` is far clearer than throwing a generic `Exception` ('not enough money'). Extend `Exception` for checked (callers must handle) or `RuntimeException` for unchecked (optional handling). Add fields for structured data that the catch block can use to take a specific action.

```
public class InsufficientFundsException extends Exception {
    Checked — caller must handle
    private final double amountRequested;    How much was requested
    private final double currentBalance;    What is available
    private final String errorCode;        Machine-readable code for API responses

    public InsufficientFundsException(double requested, double balance) {
        super("Need Rs." + requested + " but only Rs." + balance + " available");
        Human-readable message
        this.amountRequested = requested;
        this.currentBalance = balance;
        this.errorCode = "WALLET_ERR_001";
    }

    public double getShortfall() { return amountRequested - currentBalance; }
    How much more is needed
    public String getErrorCode() { return errorCode; }
}

// Using the custom exception in Wallet
public void withdraw(double amount) throws InsufficientFundsException {
    if (amount > balance) throw new InsufficientFundsException(amount, balance);
    balance -= amount;
}
```

03 try-with-resources

Any class implementing `AutoCloseable` (`FileWriter`, `Connection`, `ResultSet`...) can be declared in the `try()` header. Java calls `close()` automatically when the block exits — whether normally or via exception. This eliminates an entire category of resource-leak bugs that are notoriously hard to find in production.

```
// Classic pattern – verbose, error-prone
FileWriter fw = null;
try { fw = new FileWriter("log.txt"); fw.write("data"); }
catch (IOException e) { e.printStackTrace(); }
finally { if (fw != null) try { fw.close(); } catch (IOException ignored) {} }

Messy

// try-with-resources (Java 7+) – clean and safe
try (FileWriter fw = new FileWriter("log.txt")) {
    Declared here — closed automatically
    fw.write("TXN001,CREDIT,5000\n");
    fw.write("TXN002,DEBIT,1200\n");
} catch (IOException e) {
    System.out.println("Write failed: " + e.getMessage());
}
// fw.close() called here — guaranteed, even on
// exception

// Multiple resources – closed in reverse order
try (FileReader fr = new FileReader("log.txt");
    BufferedReader br = new BufferedReader(fr)) {
    Opened first
    Opened second, closed first
    String line;
    while ((line = br.readLine()) != null) System.out.println(line);
} catch (IOException e) { e.printStackTrace(); }
```

TRY THIS LIVE

Create a custom `AccountFrozenException` (unchecked — extends `RuntimeException`) with a field `frozenSince` (`String`). Add a boolean `isFrozen` flag to your `Wallet` class. In `processTransaction()`, throw `InsufficientFundsException` for low balance, `IllegalArgumentException` for negative amounts, and `AccountFrozenException` if the account is frozen. Write a test in `main()` that exercises all three paths with separate try-catch blocks for each.

PRO TIP — Exception Chaining

When catching one exception and throwing a higher-level one, always pass the original as the cause: throw new `WalletException("Payment failed", originalException)`. The second argument is stored as the cause and appears in the full stack trace. Without chaining, you lose the root cause — the low-level `SQLException` or `IOException` that started the chain — making debugging in production nearly impossible.

CASE STUDY — BigBasket: Custom Exception Hierarchy for Order Failures

Context — BigBasket, founded in Ahmedabad, is India's largest online grocery platform (now part of Tata Digital), processing millions of orders daily across 30+ cities.

Challenge — Distinguish dozens of distinct order failure scenarios — out of stock, minimum order value not met, delivery pincode not serviceable, payment declined, slot unavailable — each needing a specific user message and retry path.

What they did — BigBasket defines a custom exception hierarchy: `OrderException` as the base, with specific children `OutOfStockException` (carries available quantity), `SlotUnavailableException` (carries next available slot), `PincodeNotServiceableException` (carries nearest serviceable pincode), and `MinOrderValueException` (carries the gap amount). Each carries structured data the UI uses for a specific, actionable error message.

Result — Users see 'Only 2 kg Amul Milk available — we've adjusted your quantity' instead of 'Order failed'. The specific exception type drives the exact recovery action with no extra if-else in the UI layer.

Takeaway — Custom exception types are the vocabulary of your application's error language. The richer and more specific that vocabulary, the smarter your system's failure handling — and the better the user experience when things go wrong.

Source: BigBasket Engineering Blog, engineering.bigbasket.com (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lecture 25: Advanced Exception Handling, Custom Exceptions, try-with-resources (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can handle errors professionally with a custom exception hierarchy. Next, persist data beyond the program's lifetime using Java File I/O.

Explore deeper — Creating Custom Exceptions — docs.oracle.com/javase/tutorial/essential/exceptions/creating.html

Try on your own — Build a full exception hierarchy for the wallet: `WalletException` (base, checked) -> `InsufficientFundsException`, `InvalidAmountException`, `AccountSuspendedException`. Test all three with separate catch blocks. Observe how exception chaining preserves the original stack trace.

Connect the dots — Session 15 introduces File I/O — reading and writing transaction logs to disk, which requires try-with-resources to guarantee file handles are closed correctly.

Going further — Look up multi-catch syntax (`catch (IOException | SQLException e)`) — catches multiple exception types in one block with one handler, useful when the recovery action is identical for several failure modes.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java mentor. Explain why creating a custom `InsufficientFundsException` is better than throwing a generic `Exception` with a message string, in simple Hinglish. What extra information can the custom exception carry? How does it let the calling code take a more specific action? Keep it under 150 words.

SESSION

15

60 MIN · CORE JAVA

File I/O

◆ Learning Objectives

- Distinguish byte streams (InputStream/OutputStream) from character streams (Reader/Writer)
- Write text to a file using BufferedWriter in append mode
- Read text from a file line by line using BufferedReader
- Use try-with-resources to guarantee file handles are closed correctly
- Save and reload wallet transaction logs, explaining why file I/O simulates simple persistence

01 Why File I/O?

All data in your programs so far disappears the moment the program ends — it exists only in RAM. File I/O lets you write data to disk so it survives between runs: log a transaction today and read it back tomorrow. This is the simplest form of persistence and the conceptual bridge to understanding why databases exist — files handle sequential text data, but databases add querying, concurrent access, and transactional safety.

CONCEPT — Byte Streams vs Character Streams

Byte streams (InputStream, OutputStream) read and write raw bytes — use them for images, audio, compiled binaries, PDFs. Character streams (Reader, Writer) wrap byte streams with character encoding (UTF-8, UTF-16) — use them for text files, CSV logs, and configuration files. For transaction logs in text format, always use character streams. For saving a profile photo, use byte streams.

02 Writing Text — FileWriter + BufferedWriter

FileWriter opens a file and writes characters. Wrapping it in a BufferedWriter batches small writes into a buffer before flushing to disk — dramatically faster than flushing every character. Passing true as the second argument to FileWriter enables append mode: new lines are added at the end rather than overwriting the file.

```
import java.io.*;

// Append transactions to log file
try (BufferedWriter bw = new BufferedWriter(    try-with-resources — auto-close guaranteed
    new FileWriter("wallet_log.txt", true))) {
    true = append mode

    bw.write("TXN001,CREDIT,5000.00,2024-01-15 10:30:00");
    CSV-format log entry
```

```

        bw.newLine();
        bw.write("TXN002,DEBIT,1200.00,2024-01-15 11:15:00");
        bw.newLine();
        bw.write("TXN003,CREDIT,3000.00,2024-01-15 14:00:00");
        bw.newLine();
        System.out.println("Log written.");
    } catch (IOException e) {
        System.out.println("Write failed: " + e.getMessage());
    }

```

OS-correct newline (\r\n on Windows, \n on Linux)

Checked — must handle

bw.flush() + bw.close() called automatically here

03 Reading Text — FileReader + BufferedReader

FileReader opens an existing file. BufferedReader adds readLine() — which reads one complete text line per call and returns null at end-of-file. This null-sentinel while loop is the universal idiom for reading text files in Java. Java 11 also offers Files.readAllLines(path) for loading an entire small file into a List<String> in one call.

```

// Read and print the transaction log
try (BufferedReader br = new BufferedReader(
    new FileReader("wallet_log.txt"))) {
    String line;
    int count = 0;
    double runningBalance = 10000;

    System.out.println("--- Wallet Statement ---");
    while ((line = br.readLine()) != null) {
        String[] parts = line.split(",");
        double amount = Double.parseDouble(parts[2]);
        if (parts[1].equals("CREDIT")) runningBalance += amount;
        else runningBalance -= amount;
        System.out.printf("%-8s %-7s %9.2f  Balance: %10.2f%n",
            parts[0], parts[1], amount, runningBalance);
        count++;
    }
    System.out.println("Total transactions: " + count);
} catch (FileNotFoundException e) {
    System.out.println("Log not found.");
} catch (IOException e) {
}

```

Throws FileNotFoundException if file missing

Starting balance

null signals end of file

Split CSV: [txnId, type, amount, date]

Specific — file doesn't exist

Other I/O error

04 Byte Streams — FileInputStream & FileOutputStream

Byte streams handle binary data. `FileOutputStream` writes raw bytes; `FileInputStream` reads them. The standard copy-pattern uses a `byte[]` buffer to transfer chunks at a time rather than one byte at a time — critical for performance on large files. Always wrap in a buffered stream for efficiency.

```
// File copy using byte streams
try (FileInputStream fis = new FileInputStream("profile.jpg");
    Source
    FileOutputStream fos = new FileOutputStream("profile_backup.jpg")) {
    Destination
    byte[] buffer = new byte[4096];           4 KB buffer — read/write in chunks
    int bytesRead;
    while ((bytesRead = fis.read(buffer)) != -1) {
        read() returns -1 at end of stream
        fos.write(buffer, 0, bytesRead);      Write exactly what was read
    }
    System.out.println("Backup created.");
} catch (IOException e) {
    System.out.println("Copy failed: " + e.getMessage());
}
```

TRY THIS LIVE

Build a `TransactionLogger` class with two methods: (1) `log(String txnId, String type, double amount)` — appends one CSV line to 'wallet_log.txt'; (2) `printStatement()` — reads the file, splits each line, computes a running balance starting at Rs. 10000, and prints a formatted statement. Call `log()` 5 times with mixed CREDITS and DEBITS, then call `printStatement()`.

“Every database is, at its core, a sophisticated file system. Learning File I/O means learning the foundation of all persistence.”

CASE STUDY — Swiggy: File-Based Audit Logs for Order State Tracking

Context — Swiggy, the Bengaluru-based food delivery platform, processes over 1.5 million orders daily across 500+ Indian cities.

Challenge — Maintain an immutable audit trail of every order state change (placed -> confirmed -> preparing -> picked -> delivered) for dispute resolution and compliance — even during database outages.

What they did — Each order state transition is appended to a daily log file using `BufferedWriter` in append mode. The logs are continuously shipped to a centralised log aggregation system. Because the file write is

independent of the database write, the audit trail survives even when the primary database is temporarily unreachable.

Result — Swiggy reconstructs the complete timeline of any order in seconds using audit logs — essential for resolving 'order never arrived' disputes and for FSSAI regulatory audits.

Takeaway — Even in systems backed by enterprise databases, file I/O remains essential for audit logs, exports, and configuration. It is not a beginner-only concept — it is a production engineering tool.

Source: Swiggy Engineering Blog, bytes.swiggy.com (2023)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lecture 26: File I/O — Streams, Reader, Writer, Buffered Classes (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can persist data to disk and read it back. Next, learn the Collections Framework — dynamic in-memory structures that hold and process objects before they reach the database.

Explore deeper — Java I/O Tutorial — docs.oracle.com/javase/tutorial/essential/io/

Try on your own — Extend your TransactionLogger: after printing the statement, write a summary line (total CREDITS, total DEBITS, final balance) to a separate 'wallet_summary.txt' file. Use try-with-resources for both file operations.

Connect the dots — Session 16 introduces ArrayList and HashSet — the two most common in-memory collection types, which together replace most use cases that beginners reach for arrays to solve.

Going further — Explore `java.nio.file.Files` and `java.nio.file.Paths` (Java NIO2) — the modern replacement for `java.io` with cleaner syntax, atomic operations, and much better support for large file processing.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java tutor. Explain the difference between `FileWriter` and `BufferedWriter` in simple Hinglish. Use the analogy of writing a letter: `FileWriter` = run to the post box after every single word (very slow). `BufferedWriter` = write the whole letter first, then post it once (fast). Why should you always use `BufferedWriter`? Keep it under 150 words.

SESSION

16

60 MIN · CORE JAVA

Collection Framework – Part 1

◆ Learning Objectives

- Explain why the Collections Framework exists and what problems it solves over arrays
- Describe the List, Set, and Map interfaces and when to use each
- Use ArrayList to store, access, add, remove, and iterate transaction records
- Use HashSet to store unique values and perform O(1) duplicate checks
- Connect in-memory collections to the concept of pre-database storage in a service layer

01 Why Collections? Arrays Are Not Enough

Arrays have two hard limitations: fixed size (set at creation, never changes) and no built-in operations for searching, sorting, or removing elements. In a real wallet app, you cannot know how many transactions a user will make. The Java Collections Framework provides dynamic, resizable, feature-rich data structures — backed by well-tested algorithms — so you never implement your own dynamic array or hash table from scratch.

CONCEPT — Collections as In-Memory Storage Before the Database

Before data is saved to MySQL, it lives in a Collection in your service layer. An `ArrayList<Transaction>` in Java is exactly like a database result set: an ordered list of records you can iterate, search, and filter. When you learn JDBC (Module 2), you will read every row from a `ResultSet` and add it to an `ArrayList`. The `ArrayList` is the in-memory mirror of a query result — understanding one makes the other intuitive.

02 Collections Framework — Key Interfaces

Interface	Top Implementations	Ordered?	Duplicates Allowed?	Best Use Case
List	ArrayList, LinkedList	Yes (insertion order)	Yes	Transaction history, ordered item lists
Set	HashSet, LinkedHashSet, TreeSet	No / Sorted (Tree)	No	Unique device IDs, processed txn IDs
Map	HashMap, LinkedHashMap, TreeMap	No / Insertion / Sorted	Keys unique	ID -> object lookup, config settings
Queue	LinkedList, PriorityQueue	Yes (FIFO / priority)	Yes	Notification queues, task scheduling

CONCEPT — Generics (<T>) for Type Safety

`ArrayList<Transaction>` means the list accepts only `Transaction` objects. Without generics (a raw `ArrayList`), you could add any object and get a `ClassCastException` at runtime when retrieving. With generics, the type mismatch is caught at compile time — before the code ever runs. Always specify the type in angle brackets. Code to the interface: `List<String> list = new ArrayList<>()`, not `ArrayList<String> list = new ArrayList<>()` — this makes switching implementations trivial.

03 ArrayList — Dynamic Ordered List

```
import java.util.ArrayList; import java.util.List;

List<String> txns = new ArrayList<>();           Dynamic list — grows as needed

// Add
txns.add("TXN001 | CREDIT | Rs.5000");         Append to end
txns.add("TXN002 | DEBIT | Rs.1200");
txns.add("TXN003 | CREDIT | Rs.3000");
txns.add(0, "OPENING BALANCE | Rs.10000");     Insert at index 0 — shifts rest right

// Access
System.out.println(txns.get(1));                O(1) index access
System.out.println("Size: " + txns.size());     size() not .length for collections
System.out.println(txns.contains("TXN002 | DEBIT | Rs.1200"));
true

// Iterate
for (String t : txns) { System.out.println(t); }
Enhanced for-each

// Update & Remove
txns.set(1, "TXN001 | CREDIT | Rs.5500");       Replace element at index 1
txns.remove("TXN002 | DEBIT | Rs.1200");        Remove by value (first match)
txns.remove(0);                                 Remove by index
```

04 HashSet — No Duplicates, O(1) Lookup

`HashSet` stores unique values using hashing — internally like a `HashMap` with only keys. Adding a duplicate is silently ignored. `contains()`, `add()`, and `remove()` are all $O(1)$ average. The canonical use case in fintech: tracking processed transaction IDs to prevent double-processing. The canonical use case in any app: checking 'have I seen this item before?' in $O(1)$ without a linear scan.

```
import java.util.HashSet; import java.util.Set;

Set<String> processedIds = new HashSet<>();    Only unique transaction IDs

processedIds.add("TXN001");                  Added
processedIds.add("TXN002");                  Added
processedIds.add("TXN001");                  Duplicate — silently ignored, size stays 2
System.out.println(processedIds.size());      Prints 2
System.out.println(processedIds.contains("TXN001"));
    true — O(1)

// Real use: idempotent payment processing
String incoming = "TXN003";                  New payment request
if (processedIds.contains(incoming)) {
    System.out.println("Duplicate payment — rejected: " + incoming);
} else {
    processedIds.add(incoming);               Mark as processed
    System.out.println("Processing: " + incoming);
}
```

TRY THIS LIVE

Create a `List<String>` of 6 transaction log strings. Print all using for-each. Remove the 3rd transaction by value. Print the updated list. Then create a `Set<String>` of payment modes used: add 'UPI', 'CARD', 'UPI', 'WALLET', 'CARD', 'UPI'. Print the set and its size — observe only 3 unique modes remain. Check whether 'CRYPTO' is in the set.

ArrayList

Ordered (insertion order). Allows duplicates. $O(1)$ index access, $O(n)$ search. Use for history, item lists, ordered results, and anywhere the sequence matters.

HashSet

No guaranteed order. No duplicates — extras silently dropped. $O(1)$ add, contains, remove. Use for unique-value tracking, visited-ID caches, and deduplication.

CASE STUDY — Urban Company: Collections Powering Service Booking Management

Context — Urban Company (formerly UrbanClap), based in Gurugram, manages beauty, repair, and cleaning bookings across major Indian cities including Ahmedabad and Surat.

Challenge — Maintain a growing list of bookings per professional throughout the day, and check scheduling conflicts instantly when new bookings arrive — without a database round-trip per check.

What they did — Each professional's current-day schedule is a `List<Booking>` in memory (backed by the database). A `Set<String>` of booked time slot strings ('2024-01-15-14:00') enables $O(1)$ conflict checking — before confirming a new booking, `contains()` checks the Set in microseconds rather than querying the

database.

Result — Booking confirmation or rejection happens in under 200ms, even when thousands of simultaneous bookings arrive during a sale campaign — no database round-trips for conflict checking.

Takeaway — ArrayList for ordered booking history, HashSet for instant uniqueness checks — the combination of these two collections handles the two most common data management tasks in any marketplace application.

Source: Urban Company Engineering Blog, tech.urbancompany.com (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 27-28: Collections Framework — List and Set (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can use dynamic lists and unique sets. Next, learn HashMap for key-based lookup, and sorting collections with Comparators.

Explore deeper — Java Collections Tutorial — docs.oracle.com/javase/tutorial/collections/

Try on your own — Replace your wallet app's `int[]` transaction array with an `ArrayList<String>`. Add 8 transactions, remove one by value, then iterate and print. Use a `Set<String>` to track unique payment modes used across all transactions and print the count.

Connect the dots — Session 17 covers HashMap and LinkedHashMap — key-value structures that let you look up any record by its ID in $O(1)$, plus sorting collections using Comparators.

Going further — Explore `java.util.Collections` (utility class — not the interface): it provides `sort()`, `shuffle()`, `reverse()`, `min()`, `max()`, `frequency()`, and `unmodifiableList()` — ready-made algorithms for any List, usable immediately with zero extra code.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java mentor. Explain the difference between ArrayList and HashSet in simple Hinglish using a school register analogy. ArrayList = class attendance register (ordered, same student can be marked twice by mistake). HashSet = ink stamp on a pass (stamp the same name twice and it still counts as once — no duplicates). Keep it under 150 words.

SESSION

17

60 MIN · CORE JAVA

Collection Framework – Part 2

◆ Learning Objectives

- Use HashMap for $O(1)$ key-based lookup and understand its internal hashing
- Distinguish HashMap (unordered) from LinkedHashMap (insertion-ordered) and TreeMap (sorted)
- Iterate over a Map using `entrySet()`, `keySet()`, and `values()`
- Sort a List using `Collections.sort()` with natural order and custom Comparators
- Build a transaction ID-to-details map that simulates an in-memory database index

01 HashMap — Key-Value Lookup in $O(1)$

HashMap stores key-value pairs where each key maps to exactly one value. Keys must be unique; values can repeat. Any key lookup — `get()`, `containsKey()`, `remove()` — is $O(1)$ average, regardless of map size. Use HashMap whenever you need to 'find record X by its ID' — wallet by `walletId`, user by email, product by barcode. This is the in-memory equivalent of a database primary-key index.

CONCEPT — Map Operations Mirror SQL CRUD

`map.put(id, obj)` -> INSERT: add a new record keyed by ID. `map.get(id)` -> SELECT WHERE `id = ?`: retrieve by primary key in $O(1)$. `map.put(id, updatedObj)` on existing key -> UPDATE: replaces the value. `map.remove(id)` -> DELETE. `map.containsKey(id)` -> EXISTS check. Once you see HashMap as an in-memory indexed table, JDBC's `ResultSet-to-Map` pattern becomes immediately obvious.

02 HashMap Operations

```
import java.util.HashMap; import java.util.Map;

Map<String, Double> txnAmounts = new HashMap<>();
    Key: txnId (String), Value: amount (Double)

txnAmounts.put("TXN001", 5000.0);           INSERT
txnAmounts.put("TXN002", 1200.0);
txnAmounts.put("TXN003", 3000.0);
txnAmounts.put("TXN001", 5500.0);           UPDATE — same key, value replaced (5000->5500)

txnAmounts.get("TXN002")                   -> 1200.0 (O(1) SELECT)
txnAmounts.getDefault("TXN999", 0.0)       -> 0.0 (null-safe — avoids NPE)
txnAmounts.containsKey("TXN001")           -> true
```

```
txnAmounts.containsValue(3000.0)    -> true (O(n) — scans all values)
txnAmounts.remove("TXN003")        DELETE entry
txnAmounts.size()                   -> 2 (after remove)
```

03 Iterating Over a Map

Maps are not Iterable directly — you iterate over their key set, value collection, or entry set. `entrySet()` is the most efficient approach: each `Map.Entry` gives you both key and value in one object, avoiding a second `get()` call per key. Java 8+ also supports `forEach()` with a lambda.

```
// Iterate all key-value pairs — most common
for (Map.Entry<String, Double> entry : txnAmounts.entrySet()) {
    System.out.printf("%-8s -> Rs.%9.2f\n", entry.getKey(), entry.getValue());
}

// Sum all values
double total = 0;
for (double amt : txnAmounts.values()) { total += amt; }
System.out.println("Grand total: Rs." + total);

// Java 8 forEach with lambda (preview — revisit after learning lambdas)
txnAmounts.forEach((id, amt) -> System.out.println(id + " = " + amt));
```

04 LinkedHashMap, TreeMap & Sorting Collections

`LinkedHashMap` maintains insertion order — the iteration sequence matches the `put()` sequence. `TreeMap` sorts keys in natural order (alphabetical for Strings, ascending for numbers). For sorting Lists, `Collections.sort()` sorts in natural order; pass a `Comparator` for custom ordering. Java 8 lambda syntax makes `Comparators` concise.

```
import java.util.*;

// LinkedHashMap — preserves insertion order
Map<String, Double> orderedLog = new LinkedHashMap<>();
orderedLog.put("TXN003", 3000.0);           Added first
orderedLog.put("TXN001", 5000.0);
orderedLog.put("TXN002", 1200.0);
// Iteration: TXN003 -> TXN001 -> TXN002 (insertion order)

// Sort a List of amounts
List<Double> amounts = new ArrayList<>(Arrays.asList(3000.0, 5000.0, 1200.0, 800.0));
```

```

Collections.sort(amounts);                                Ascending: [500.0, 800.0, 1200.0, 3000.0]
Collections.sort(amounts, Collections.reverseOrder());
    Descending: [3000.0, 1200.0, 800.0, 500.0]

// Sort txn IDs alphabetically using lambda Comparator
List<String> ids = Arrays.asList("TXN003", "TXN001", "TXN002");
ids.sort((a, b) -> a.compareTo(b));                      Lambda Comparator — sorts ascending
System.out.println(ids);                                  [TXN001, TXN002, TXN003]

// TreeMap — keys automatically sorted
Map<String, Double> sortedMap = new TreeMap<>(txnAmounts);
    Keys in alphabetical order on iteration

```

TRY THIS LIVE

Create a `Map<String, Double>` of 5 transaction IDs to amounts. Use `entrySet()` to print each. Find the transaction with the highest amount by iterating over `values()` and tracking the max. Then sort the amounts as a `List` (ascending and descending). Finally, copy the `HashMap` into a `LinkedHashMap` and a `TreeMap` — compare the iteration order of all three.

PRO TIP — `HashMap` vs `LinkedHashMap` vs `TreeMap`

`HashMap`: fastest $O(1)$ operations, no order — use by default. `LinkedHashMap`: $O(1)$ operations + insertion order preserved — use for LRU caches, recent-transaction feeds. `TreeMap`: $O(\log n)$ operations, keys sorted — use for a sorted leaderboard, range-query-like traversal, or alphabetically sorted configs. For 95% of use cases, `HashMap` is the answer. Switch to the others only when order matters.

CASE STUDY — Nykaa: `HashMap` for Product Catalogue Lookup

Context — Nykaa, India's leading beauty and lifestyle e-commerce platform (headquartered in Mumbai), manages over 4 million SKUs across its app and website.

Challenge — Serve product details for any SKU in under 10 ms during peak sale traffic — when tens of thousands of users browse simultaneously.

What they did — Product metadata is loaded at startup into a `Map<String, Product>` (SKU -> Product object) in the service layer. Any product lookup is an $O(1)$ `HashMap.get()` call — no database round-trip. A `LinkedHashMap` per session tracks recently viewed SKUs in insertion order for the 'Recently Viewed' widget.

Result — Nykaa's product pages load in under 300 ms even during major sale events, with `HashMap` lookups serving the majority of product queries without touching the database.

Takeaway — `HashMap` is the data structure behind every 'look up X by its ID' feature in every e-commerce app — product by SKU, user by ID, transaction by reference. It is the in-memory equivalent of a database primary-key index, and mastering it unlocks a huge range of real engineering patterns.

Source: Nykaa Engineering Blog, tech.nykaa.com (2023)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lectures 29-30: HashMap, LinkedHashMap, TreeMap and Sorting (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You now command all major Java collection types. The final session introduces threads and wraps up the full Core Java module.

Explore deeper — Java Map Interface — docs.oracle.com/javase/tutorial/collections/interfaces/map.html

Try on your own — Build a WalletLedger: `Map<String, List<Double>>` where each key is a date string and the value is the day's transaction amounts. Add transactions for 3 days. Use `entrySet()` to iterate — for each day print all amounts and the day's total. Sort each day's list in descending order before printing.

Connect the dots — Session 18 introduces multithreading and delivers the full Core Java module recap — preparing you for the Digital Wallet Console App mini project.

Going further — Explore Java 8 Streams on collections: `list.stream().filter(x -> x > 1000).sorted().collect(Collectors.toList())` — functional-style, declarative collection processing that replaces many explicit for-loops in modern Java code.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java instructor. Explain HashMap in simple Hinglish using the telephone directory analogy: the name is the key, the number is the value. You go directly to the name without reading every page (O(1) lookup). Explain put(), get(), containsKey(), and why keys must be unique. Compare HashMap and LinkedHashMap using 'random shelf' vs 'ordered shelf' in a library. Keep it under 150 words.

SESSION

18

60 MIN · CORE JAVA

Threads & Recap

◆ Learning Objectives

- Explain the thread lifecycle and distinguish a process from a thread
- Create threads by extending Thread and by implementing Runnable
- Use sleep() and join() to control thread timing and sequencing
- Build a background cashback updater that runs as a daemon thread
- Consolidate all Core Java concepts from Sessions 01-18 in a single review

01 Processes, Threads & Concurrency

A process is a running program with its own memory space. A thread is the smallest unit of execution inside a process. Every Java program starts with one main thread, but additional threads can be created to run tasks concurrently. In a real wallet backend, one thread processes a payment, another sends a confirmation SMS, and a third logs the transaction — all simultaneously, without waiting for each other to finish.

CONCEPT — Thread Lifecycle

A Java thread moves through five states: NEW (created, not yet started), RUNNABLE (running or ready), BLOCKED/WAITING (paused waiting for a lock or another thread), TIMED_WAITING (sleeping for a fixed duration via sleep()), and TERMINATED (finished). The JVM thread scheduler decides which RUNNABLE thread gets CPU time — you write the task logic in run() and call start() to hand it off to the scheduler.

02 Creating Threads — Two Approaches

Approach 1: extend Thread and override run(). Simple, but wastes the one inheritance slot Java allows. Approach 2: implement Runnable and pass it to a Thread constructor. Preferred — separates the task from the thread mechanism and works with ExecutorService thread pools used in production. Always call start() — never call run() directly, which just executes in the current thread with no concurrency.

```
// Approach 1: Extend Thread
class CashbackUpdater extends Thread {
    private int intervalSec;
    CashbackUpdater(int intervalSec) { this.intervalSec = intervalSec; }
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Cashback cycle " + i + " - " + Thread.currentThread().getName());
        }
    }
}
```

Task runs in its own thread
5 cashback update cycles

```
try { Thread.sleep(intervalSec * 1000L); }
    Pause N seconds between cycles
catch (InterruptedException e) { return; }
    Exit cleanly if interrupted
    }
    }
}
```

```
// Approach 2: Implement Runnable (PREFERRED)
class SMSNotifier implements Runnable {
    private String phone;
    SMSNotifier(String phone) { this.phone = phone; }
    @Override
    public void run() {
        System.out.println("SMS sent to " + phone + " via " +
            Thread.currentThread().getName());
    }
}
```

03 Starting, Joining & Daemon Threads

```
// Starting threads
CashbackUpdater updater = new CashbackUpdater(10);
    Update every 10 seconds
updater.setName("Cashback-Thread");           Name for easy debugging
updater.setDaemon(true);                      Daemon: JVM exits when only daemons are left
updater.start();                             Launches new OS thread, calls run() there

Thread sms = new Thread(new SMSNotifier("+919876543210"), "SMS-Thread");
sms.start();

System.out.println("Main thread continues without waiting");

// join() – make main thread wait for another thread
Thread report = new Thread(() -> {           Lambda shorthand for Runnable
    System.out.println("Generating monthly report...");
    try { Thread.sleep(2000); } catch (InterruptedException e) {}
    Simulate 2s work
    System.out.println("Report ready.");
});
report.start();
report.join();                               Main thread waits here until report thread finishes
```

```
System.out.println("Main continues – report is guaranteed ready.");
```

CONCEPT — Race Condition & synchronized

If two threads simultaneously read and modify the same field — say, both read balance = 5000 and both deduct 3000 — the final balance could be 2000 instead of -1000. This is a race condition. Mark the method with synchronized: the JVM allows only one thread inside a synchronized method at a time. For simple counters, use AtomicInteger — a thread-safe integer wrapper with no locking overhead.

TRY THIS LIVE

Create a Runnable called InterestCalculator that prints 'Calculating interest — month X' for 3 months with a 1-second pause between each. Create a Runnable called BalanceSync that prints 'Balance synced to server'. Start both as separate threads in main(). Observe that both run concurrently. Add a join() on InterestCalculator and verify that the main thread waits for it before printing 'All tasks done'.

04 Core Java Module — Full Recap

You have completed all 18 sessions of Core Java — the complete foundation for the Java Full Stack Career Program. Every concept you have learned in this module will be used in every subsequent module: OOP in Spring, exception handling in JDBC, collections in REST API responses, and threads in async Spring Boot tasks.

Sessions	Topic Group	Key Takeaway
01–02	Setup & Data	JVM ecosystem, 8 primitives, operators, type casting
03–05	Control Flow	Conditionals, loops, arrays — the logic skeleton of every program
06–09	OOP Pillars I	Classes, objects, methods, constructors, inheritance
10–11	OOP Pillars II	Polymorphism, encapsulation, abstraction, interfaces
12	Strings	Immutability, StringBuilder, formatting, masking
13–14	Exceptions	try-catch-finally, custom exceptions, try-with-resources

Sessions	Topic Group	Key Takeaway
15	File I/O	Persistence to disk, BufferedWriter/Reader, byte vs char streams
16–17	Collections	ArrayList, HashSet, HashMap, LinkedHashMap, sorting
18	Threads	Lifecycle, Runnable, sleep, join, daemon, synchronization

“Core Java is not a chapter you finish — it is a language you now speak. Every framework, every API, every system you build from here is Java beneath.”

CASE STUDY — BYJU'S / Vedantu: Threads Powering Live Class Experiences

Context — Vedantu (Bengaluru-based edtech, now merged with BYJU'S) delivered live online classes to over 35 million students across India at its peak.

Challenge — Stream live video, handle real-time chat messages, run live polls, update student attendance, and process payments simultaneously — all during a live class session with thousands of concurrent users.

What they did — Vedantu's backend uses Java multithreading heavily: one thread per live session handles WebSocket connections, a separate thread pool processes incoming chat messages, another scheduled thread updates attendance every 30 seconds, and a background daemon thread preloads the next class's resources. Collections (ConcurrentHashMap for thread-safe session state) and exception handling (recover gracefully from student disconnections) are used throughout.

Result — Vedantu sustained 99.5% uptime during live classes with 100,000+ simultaneous students, handling real-time interactions with sub-200ms latency.

Takeaway — All 18 sessions of Core Java — threads, collections, exceptions, OOP, strings — work together in every real production system. There is no 'advanced' and 'basic': it is one complete toolkit.

Source: Vedantu Engineering Blog, medium.com/vedantu-engineering (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Lecture 31: Multithreading, Runnable and Thread Lifecycle (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You have completed Core Java. You are ready to build the Digital Wallet Console App mini project and then advance to Module 2 — JDBC & Database Programming.

Explore deeper — Java Concurrency in Practice — jcip.net (the definitive book on Java threading, free to read online in excerpts)

Try on your own — Build the mini project: a Digital Wallet Console App using Scanner for input, ArrayList and HashMap for data storage, custom exceptions for error cases, a background Thread for interest calculation every 30 seconds, and FileWriter to log every transaction.

Connect the dots — Module 2 (JDBC & Database Programming) connects your Java objects to a real MySQL database — replacing the in-memory ArrayList and HashMap with persistent, queryable relational storage.

Going further — Research Java's ExecutorService and thread pool (Executors.newFixedThreadPool(n)) — the production approach to managing many threads efficiently without creating a new thread per task, which is expensive at scale.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java instructor. Explain the difference between extending Thread and implementing Runnable in simple Hinglish. Use the analogy of a restaurant: Runnable = the recipe (the task). Thread = the chef (the executor). A chef can follow any recipe — but a chef who IS a recipe can only ever make one dish. Explain why Runnable is preferred. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 1

Interview Questions

◆ Core Java

Rehearse these questions out loud, aiming for a 60-second answer each. The hint is only a nudge — try answering before you look at it. Compare your answer to the model answer afterward, not before.

1

What is the main difference between the JDK, JRE, and JVM in Java?

Think about which one is for development, which one is for running programs, and which one is the actual engine.

MODEL ANSWER

The JDK (Java Development Kit) is the full package for developers — it includes compilers, debuggers, and tools to write and build Java programs. The JRE (Java Runtime Environment) is a subset used only for running already-compiled Java applications; it does not include development tools. The JVM (Java Virtual Machine) is the engine inside the JRE that actually executes Java bytecode. In short: JDK is for building, JRE is for running, and JVM is the core execution layer inside the JRE.

2

Why is Java considered platform-independent?

Focus on the role of the JVM in running Java bytecode.

MODEL ANSWER

Java source code is compiled into an intermediate form called bytecode (.class files) rather than native machine code. This bytecode is not tied to any specific operating system or processor. The JVM, which is platform-specific (there are different JVMs for Windows, macOS, Linux), interprets and executes this bytecode. So the same compiled Java program can run on any machine that has a JVM installed — hence the slogan 'Write Once, Run Anywhere'.

3

Describe the steps to install the JDK and set up a Java development environment using Eclipse or IntelliJ.**MODEL ANSWER**

First, download the JDK from Oracle or Adoptium and run the installer. After installation, set the JAVA_HOME environment variable to the JDK directory and add the bin folder to the system PATH. Verify the installation by running `java -version` and `javac -version` in the terminal. Then download and install your IDE (Eclipse or IntelliJ IDEA), create a new Java project, and the IDE will auto-detect the installed JDK. You can then create a class with a main method and run it directly from the IDE.

4

Write a simple Java program that prints 'Hello Digital Wallet!' to the console.**MODEL ANSWER**

```
public class HelloWallet { public static void main(String[] args) { System.out.println("Hello Digital Wallet!"); } }
```

— The class name must match the file name. The main method is the entry point Java looks for when running the program. `System.out.println` writes the string to standard output followed by a newline. This is the minimal structure every Java application needs.

5

Explain the structure of a basic Java class. What is the purpose of the main method?

MODEL ANSWER

A Java class starts with an optional package declaration, then import statements, and then the class definition using the class keyword. Inside the class, you define fields (variables) and methods. The main method — `public static void main(String[] args)` — is the entry point of any standalone Java application. 'public' makes it accessible to the JVM, 'static' means it can be called without creating an object, 'void' means it returns nothing, and `String[] args` allows command-line arguments to be passed in.

6

What are primitive data types in Java? Give three examples.

Think about int, double, boolean, etc.

MODEL ANSWER

Primitive data types in Java are the most basic data types built directly into the language — they are not objects and have no methods. Java has eight primitives: `int` (32-bit integer), `double` (64-bit decimal), `boolean` (`true/false`), `char` (single character), `byte`, `short`, `long`, and `float`. For example: `int age = 25;` `double balance = 1500.75;` `boolean isActive = true;` These types are stored by value in memory, making them faster than object types.

7

How are primitive types different from non-primitive types in Java?

Consider memory usage and built-in vs user-defined types.

MODEL ANSWER

Primitive types (`int`, `double`, `boolean`, etc.) store actual values directly in memory, take up fixed amounts of space, and cannot be null. Non-primitive types (also called reference types) such as `String`, arrays, and user-defined classes store a reference (memory address) to an object, can be null, and have methods associated with them. For example, a `String` object has methods like `.length()` and `.toUpperCase()`, whereas the primitive `int` has no methods. Non-primitive types are created using the `new` keyword (except `String` literals).

8

Write a Java statement to declare a variable to store a user's wallet balance as a decimal number.

MODEL ANSWER

`double walletBalance = 0.0;` — `double` is the appropriate primitive type for decimal numbers in Java because it provides 64-bit precision, which is suitable for financial amounts (though for production finance apps, `BigDecimal` is preferred for exact precision). You could also declare it as: `double walletBalance;` and assign a value later. Using a meaningful name like `walletBalance` makes the code self-documenting.

9

What is type casting in Java? Give an example of both implicit and explicit casting.

MODEL ANSWER

Type casting is converting a value from one data type to another. Implicit casting (widening) happens automatically when converting a smaller type to a larger type without data loss — for example, `int x = 5;` `double y = x;` Java automatically converts the `int` to a `double`. Explicit casting (narrowing) requires the programmer to manually specify the target type because data could be lost — for example, `double price = 99.99;` `int rounded = (int) price;` — this truncates the decimal, giving `rounded = 99.`

10

What is the difference between the == and != operators in Java?

MODEL ANSWER

== checks if two values are equal, while != checks if two values are not equal. For primitive types like int or boolean, they compare actual values: 5 == 5 returns true. For reference types (objects), == and != compare memory addresses (references), not the actual content — so two different String objects with the same text return false with ==. To compare the content of objects, you should use the .equals() method instead. This is a common source of bugs for beginners.

11

How would you use an if-else statement in Java to check if a user's wallet balance is less than 100 and print a warning message?

Remember the syntax: *if (condition) { ... } else { ... }*

MODEL ANSWER

double balance = 80.0; if (balance < 100) { System.out.println("Warning: Low balance!"); } else { System.out.println("Balance is sufficient."); } — The if keyword is followed by a boolean condition in parentheses. If the condition is true, the block inside the first braces executes; otherwise the else block executes. You can chain multiple conditions using else if. The condition here uses the < (less than) comparison operator.

12

What is the purpose of the switch statement in Java, and when would you use it instead of multiple if-else statements?

MODEL ANSWER

The switch statement provides a cleaner way to check a single variable against multiple discrete values. Instead of writing a long chain of if-else if blocks, switch matches the variable to a case label and executes the corresponding block. It is best used when you have one variable with several fixed values to compare against, such as days of the week or transaction types. Each case should end with a break statement to prevent fall-through (executing the next case). switch is generally more readable and slightly more efficient than many if-else chains.

13

Rewrite this logic using a ternary operator: If balance < 100, print 'Low Balance', else print 'Sufficient Balance'.

The ternary operator uses the format: *condition ? value1 : value2*

MODEL ANSWER

String message = (balance < 100) ? "Low Balance" : "Sufficient Balance"; System.out.println(message); — The ternary operator is a compact alternative to if-else for simple conditional assignments. The condition comes first, followed by ?, then the value to return if true, then :, then the value if false. It is useful for short, inline decisions but should be avoided for complex logic where an if-else is more readable.

14

How can you relate the use of conditional statements in Java to SQL WHERE clauses?

MODEL ANSWER

Both Java conditionals and SQL WHERE clauses serve the same fundamental purpose: filtering data based on a condition. In SQL, WHERE balance < 100 filters rows from a table where the balance is under 100. In Java, an if (balance < 100) block filters program flow for the same condition. When you later fetch data from a database in Java, you use both: the SQL WHERE clause filters results at the database level, and Java conditionals then process or display the filtered data at the application level.

15

What is the difference between a for loop and a while loop in Java?

Think about when each loop checks its condition.

MODEL ANSWER

A for loop is used when you know exactly how many times you want to iterate — it bundles the initialization, condition check, and increment into one line: for (int i = 0; i < 10; i++). A while loop is used when the number of iterations is not known in advance and depends on a condition: while (userIsLoggedIn). A do-while loop is similar to while but guarantees the body executes at least once. In practice, for loops are common for array traversal and counting, while while loops are used for event-driven or condition-based repetition.

16

Write a Java loop that prints the numbers 1 to 5 using a for loop.

MODEL ANSWER

for (int i = 1; i <= 5; i++) { System.out.println(i); } — The loop starts i at 1, runs as long as i is less than or equal to 5, and increments i by 1 after each iteration. This prints 1, 2, 3, 4, 5 on separate lines. The for loop is ideal here because the number of iterations (5) is known ahead of time.

17

How would you use a loop to display a summary of monthly transactions stored in an array?

MODEL ANSWER

double[] transactions = {500.0, 200.0, 750.0}; for (int i = 0; i < transactions.length; i++) { System.out.println("Transaction " + (i+1) + ": " + transactions[i]); } — You access each element using its index, and transactions.length gives the number of elements so you don't need to hardcode the size. You can also use a for-each loop: for (double t : transactions) { System.out.println(t); } which is simpler when you don't need the index.

18

What is the purpose of the break and continue statements in loops?

MODEL ANSWER

break immediately exits the loop entirely, regardless of the loop condition. continue skips the rest of the current iteration and moves to the next one. For example, in a loop processing transactions, if you encounter a fraudulent transaction, you might use break to stop processing altogether. If you want to skip transactions below a minimum threshold but continue checking others, you would use continue. Both are used sparingly — overusing them can make loops harder to read and debug.

19

How do you declare and initialize an array to store the last 10 transaction amounts in Java?

Consider the syntax for creating an array of a specific size.

MODEL ANSWER

`double[] transactions = new double[10];` — This declares an array named `transactions` that holds 10 double values, all initialized to 0.0 by default. Alternatively, you can declare and initialize values directly: `double[] transactions = {100.0, 200.0, 50.0};` Arrays in Java are fixed in size once created. You access elements using a zero-based index: `transactions[0]` is the first element and `transactions[9]` is the last in a 10-element array.

20

Explain the difference between single-dimensional and multi-dimensional arrays in Java.

MODEL ANSWER

A single-dimensional array is a simple list of elements stored in sequence: `int[] marks = new int[5];` accessed by one index. A multi-dimensional array is essentially an array of arrays — a 2D array looks like a table with rows and columns: `int[][] matrix = new int[3][4];` accessed by two indices (row and column). Multi-dimensional arrays are useful for representing grids, matrices, or tabular data such as a monthly expense breakdown where rows represent months and columns represent categories.

21

Write code to traverse an array of transaction amounts and print each one.

MODEL ANSWER

`double[] amounts = {100.0, 250.5, 75.0, 500.0}; for (double amount : amounts) { System.out.println("Amount: " + amount); }` — The enhanced for-each loop is the cleanest way to traverse an array when you only need the value and not the index. If you need the index (e.g., to number transactions), use: `for (int i = 0; i < amounts.length; i++) { System.out.println("Txn " + (i+1) + ": " + amounts[i]); }`

22

How do arrays in Java help model database tables (rows)?

MODEL ANSWER

A database table row is a collection of related values for one record — for example, a transaction row has an ID, amount, and date. In Java, a 1D array can represent one column of data (all transaction amounts), while a 2D array can represent a full table where each row is a record and each column is a field. Before databases, arrays were the primary way to hold in-memory data. When you later fetch rows from a database using JDBC, you often store each column's values in arrays or lists for display or processing.

23

What is Object-Oriented Programming (OOP), and why is it preferred over procedural programming?

Think about code organization and reusability.

MODEL ANSWER

OOP is a programming paradigm that organizes code around objects, which combine data (fields) and behavior (methods) into a single unit. In procedural programming, data and functions are separate, and programs run as a sequence of instructions. OOP is preferred because it mirrors real-world entities (a User object has a name, balance, and methods like `deposit`), making code easier to model and understand. It also promotes reuse through inheritance, encapsulation to hide internal details, and polymorphism to write flexible code. These features make large applications easier to build, maintain, and extend.

24

Give a real-world analogy for objects and classes, using a User and Wallet as examples.

MODEL ANSWER

A class is like a blueprint or template — it defines the structure and behavior without being a real thing itself. An object is a specific instance created from that blueprint. For example, the User class defines that every user has a name, email, and balance. When you create `User u = new User();` you get a specific user object with actual values. Similarly, the Wallet class defines that a wallet has an owner and a balance with deposit/withdraw methods — each User can have their own Wallet object with its own state.

25

How do you define a class in Java, and how do you create an object of that class?

MODEL ANSWER

A class is defined using the class keyword: `public class Wallet { double balance; void deposit(double amount) { balance += amount; } }` — To create an object, you use the new keyword along with the class constructor: `Wallet myWallet = new Wallet();` — This allocates memory for the object. You then access its fields and methods using the dot operator: `myWallet.deposit(500);` — The variable `myWallet` holds a reference to the object in memory.

26

What is the difference between instance variables and static variables in a class?

MODEL ANSWER

Instance variables belong to a specific object — each object gets its own copy. For example, every Wallet object has its own balance. They are declared inside the class but outside any method, without the static keyword. Static variables belong to the class itself and are shared across all objects of that class — for example, a static `int walletCount` could track how many wallets have been created. Changing a static variable from one object changes it for all. Static variables are accessed via the class name: `Wallet.walletCount`.

27

Write a method in a Wallet class called `addMoney` that takes an amount as a parameter and updates the balance.

MODEL ANSWER

`public class Wallet { private double balance; public void addMoney(double amount) { if (amount > 0) { balance += amount; } } public double getBalance() { return balance; } }` — The method takes a double parameter, validates that it is positive (good defensive practice), then adds it to the instance variable `balance`. Private access on `balance` enforces encapsulation — the balance can only be modified through controlled methods like `addMoney`.

28

How are classes in Java similar to tables in a database, and how are methods similar to CRUD operations?

MODEL ANSWER

A class in Java maps to a table in a database — the class name corresponds to the table name, and the class fields correspond to columns. Each object instance is like a row in that table. Methods in the class correspond to CRUD operations: a `saveUser()` method is like `INSERT`, `getUser()` is like `SELECT`, `updateBalance()` is like `UPDATE`, and `deleteUser()` is like `DELETE`. This mapping is the foundation of ORM frameworks like Hibernate, which automatically bridge Java objects and database tables.

29

What is a constructor in Java? How does it differ from a regular method?

Think about how and when each is called.

MODEL ANSWER

A constructor is a special method that is automatically called when an object is created with new. It has the same name as the class and no return type (not even void). Its purpose is to initialize the object's state. A regular method has a different name, has a return type, and must be called explicitly. For example: `User u = new User("Ananya", 500.0);` calls the constructor with parameters, initializing the name and balance fields. If you do not define a constructor, Java provides a default no-argument constructor automatically.

30

Write two constructors for a User class: one with no parameters and one that takes a name and balance.

MODEL ANSWER

```
public class User { String name; double balance; public User() { this.name = "Unknown"; this.balance = 0.0; }
public User(String name, double balance) { this.name = name; this.balance = balance; } }
```

— The no-arg constructor sets default values. The parameterized constructor takes inputs to initialize the object meaningfully. Having both gives flexibility: `User u1 = new User();` and `User u2 = new User("Priya", 1000.0);` are both valid.

31

Explain method overloading with an example.

MODEL ANSWER

Method overloading allows a class to have multiple methods with the same name but different parameter lists (different number, types, or order of parameters). Java decides which version to call based on the arguments you pass at compile time. For example, in a Wallet class: `void deposit(double amount)` and `void deposit(double amount, String note)` are two overloaded versions. Calling `wallet.deposit(500.0)` uses the first, while `wallet.deposit(500.0, "Birthday gift")` uses the second. The return type alone cannot distinguish overloaded methods.

32

How is method overloading similar to having different forms of INSERT queries in a database?

MODEL ANSWER

Just as you can INSERT into a table with a different number of specified columns — `INSERT INTO users(name) VALUES('Raj')` vs `INSERT INTO users(name, balance) VALUES('Raj', 500)` — method overloading lets you call the same method name with different data. Both provide flexibility: use what you have and let the system handle the rest with defaults. In Java, overloaded methods serve the same operation (like adding a transaction) but adapt to different amounts of available information, mirroring how SQL lets you omit optional columns.

33

What is inheritance in Java? Give an example using User and PremiumUser classes.

MODEL ANSWER

Inheritance allows one class (subclass) to acquire the fields and methods of another class (superclass) using the extends keyword. This promotes code reuse. For example: `public class PremiumUser extends User { double cashbackRate; void applyCashback() { balance += balance * cashbackRate; } }` — PremiumUser inherits all fields (name, balance) and methods from User, and adds its own new field and behavior. This means PremiumUser objects can do everything a User can, plus more. The is-a relationship applies: a PremiumUser is a User.

34

What is the purpose of the super and this keywords in Java inheritance?

One refers to the parent class, the other to the current instance.

MODEL ANSWER

this refers to the current object instance and is used to distinguish between instance variables and local variables with the same name: `this.name = name;` It can also be used to call another constructor in the same class: `this()`. super refers to the parent class and is used to call the parent's constructor: `super(name, balance);` or to access an overridden parent method: `super.toString()`. `super()` must be the first statement in a constructor when used. Together, these keywords help manage the relationship between a class and its parent during object construction and method calls.

35

Explain the difference between single, multilevel, and hierarchical inheritance in Java.

MODEL ANSWER

Single inheritance: one class extends one parent — PremiumUser extends User. Multilevel inheritance: a chain where each class extends the one above — Employee extends Person, then Manager extends Employee. Hierarchical inheritance: multiple classes extend the same parent — SavingsAccount and CurrentAccount both extend BankAccount. Java does not support multiple inheritance (a class extending two parents directly) because it causes ambiguity. To achieve multiple inheritance behavior, Java uses interfaces instead.

36

What is polymorphism in Java? Distinguish between compile-time and runtime polymorphism.

MODEL ANSWER

Polymorphism means 'many forms' — the same method name can behave differently depending on context. Compile-time polymorphism (static binding) is achieved through method overloading: the correct method is chosen at compile time based on the parameter types. Runtime polymorphism (dynamic binding) is achieved through method overriding: a subclass provides its own version of a parent method, and the correct version is determined at runtime based on the actual object type. For example: `Payment p = new UPIPayment(); p.process();` — Java calls UPIPayment's process() at runtime, even though p is declared as type Payment.

37

How does method overriding work in Java? Provide a simple example.

MODEL ANSWER

Method overriding occurs when a subclass defines a method with the exact same name, return type, and parameters as a method in its superclass. The `@Override` annotation is recommended to confirm the intent. Example: `class Animal { void sound() { System.out.println("Generic sound"); } }` `class Dog extends Animal { @Override void sound() { System.out.println("Bark"); } }` — When you call `sound()` on a `Dog` object, Java executes `Dog`'s version. The parent version can still be accessed with `super.sound()`.

38

Describe a scenario where you would use dynamic method dispatch.

MODEL ANSWER

Dynamic method dispatch lets you hold a reference of the parent type that actually points to a child object, and Java calls the correct overridden method at runtime. A practical scenario: you have a `Payment` abstract class with subclasses `UPIPayment`, `CardPayment`, and `NetBanking`. You can write: `Payment p = new UPIPayment(); p.process();` — At runtime, Java dispatches to `UPIPayment`'s `process()` method. This allows you to write generic code that works for any payment type without knowing the exact type at compile time, which is the foundation of polymorphic design.

39

What is encapsulation, and how do access modifiers help achieve it in Java?

Think about data hiding and controlling access.

MODEL ANSWER

Encapsulation is the OOP principle of hiding an object's internal data and only exposing it through controlled methods. This protects the object from invalid states. Access modifiers control visibility: `private` means only the class itself can access the member; `protected` means the class and its subclasses; `public` means accessible everywhere. In a `Wallet` class, declaring `balance` as `private` prevents external code from directly setting it to a negative value. The only way to change `balance` is through `deposit()` and `withdraw()` methods, which can include validation logic.

40

How do you use getters and setters to access private variables in a class?

MODEL ANSWER

Getters are public methods that return the value of a private field: `public double getBalance() { return balance; }` Setters are public methods that set the value, typically with validation: `public void setBalance(double balance) { if (balance >= 0) { this.balance = balance; } }` — By making fields `private` and exposing only getters/setters, you enforce encapsulation. IDEs like IntelliJ can auto-generate these. The naming convention is `getFieldName()` for getters and `setFieldName()` for setters.

41

What is the difference between an abstract class and an interface in Java?**MODEL ANSWER**

An abstract class can have both abstract methods (no body) and concrete methods (with body), can have fields, and a class can extend only one abstract class. An interface traditionally declared only abstract methods (Java 8+ allows default and static methods) and constants, and a class can implement multiple interfaces. Use an abstract class when you want to share code among related classes. Use an interface when you want to define a contract that unrelated classes can fulfill. For example, Flyable could be an interface implemented by both Airplane and Bird.

42

Write an abstract class called Payment with an abstract method processPayment.**MODEL ANSWER**

```
public abstract class Payment { private double amount; public Payment(double amount) { this.amount = amount; } public double getAmount() { return amount; } public abstract void processPayment(); } — The abstract keyword on the class means it cannot be instantiated directly. The abstract method processPayment() has no body — each subclass (e.g., UPIPayment, CardPayment) must provide its own implementation. The class can still have concrete methods and fields that all subclasses inherit.
```

43

How does abstraction in Java help define API contracts between different layers of an application?**MODEL ANSWER**

Abstraction hides implementation details and exposes only what is necessary. When one layer of an application depends on an abstract class or interface rather than a concrete class, the layers are loosely coupled. For example, a service class can depend on a PaymentProcessor interface without knowing whether it is UPI or card-based. This means you can switch implementations without changing the service. In REST APIs, the controller layer calls service methods through interfaces, and in databases, DAOs implement repository interfaces — the calling code never needs to know the internals.

44

What is the difference between String, StringBuilder, and StringBuffer in Java?*Consider mutability and thread safety.***MODEL ANSWER**

String is immutable — every modification creates a new object, which is memory-inefficient in loops. StringBuilder is mutable — it builds strings in place and is the preferred choice for string operations in a single-threaded context because it is fast. StringBuffer is also mutable but is synchronized (thread-safe), making it suitable for multi-threaded use but slightly slower than StringBuilder. For most use cases like building a string in a loop, use StringBuilder. For thread-safe scenarios, use StringBuffer. For constant strings, use String.

45

Write code to mask the first four characters of a wallet ID (e.g., 'WAL1234' -> '***1234').

MODEL ANSWER

String walletId = "WAL1234"; String masked = "****" + walletId.substring(3); System.out.println(masked); // Output: ***1234 — substring(3) returns the string starting from index 3 (zero-based), so it captures '1234'. You concatenate three asterisks to replace the first three characters. If the number of masked characters matches the prefix length, you could use: String masked = "".repeat(3) + walletId.substring(3); for a more dynamic version.

46

How would you compare two strings for equality in Java?

MODEL ANSWER

Use the .equals() method to compare string content: str1.equals(str2) returns true if both strings have the same characters in the same order. Do not use == for string comparison — it checks whether both variables point to the same memory address (reference equality), not the same content. Two String objects with identical text created separately with new will fail == but pass .equals(). For case-insensitive comparison, use .equalsIgnoreCase(). For null-safe comparison, use Objects.equals(str1, str2).

47

Why is exception handling important in Java applications?

MODEL ANSWER

Exception handling prevents a program from crashing unexpectedly and allows it to recover gracefully from errors. In a production application, unhandled errors can corrupt data, expose system details to users, or bring down the service. Java's try-catch-finally mechanism lets you catch errors, log them, display a user-friendly message, and continue execution. It also separates the normal flow from error-handling code, making programs easier to read. Checked exceptions (like IOException) force developers to anticipate and handle predictable failures.

48

Write a try-catch block that catches an exception if a user tries to withdraw more than their wallet balance.

MODEL ANSWER

```
try { if (amount > balance) { throw new Exception("Insufficient funds."); } balance -= amount;
System.out.println("Withdrawal successful."); } catch (Exception e) { System.out.println("Error: " +
e.getMessage()); } — You explicitly throw an exception when the business rule is violated. The catch block
handles it and prints an error message instead of crashing. In practice, you would create a custom
InsufficientFundsException instead of using the generic Exception class.
```

49

What is the purpose of the finally block in exception handling?

MODEL ANSWER

The finally block contains code that always executes after the try-catch, whether an exception occurred or not. It is typically used for cleanup operations — closing file streams, releasing database connections, or unlocking resources — that must happen regardless of success or failure. For example: finally { connection.close(); } — Even if a SQL exception is thrown in the try block, the connection will still be closed. Java 7+ introduced try-with-resources as a cleaner alternative for auto-closing resources.

50

Explain the difference between throw and throws in Java.

MODEL ANSWER

throw is used inside a method to actually raise an exception: `throw new InsufficientFundsException("Not enough balance");` — it creates and fires the exception object. throws is used in the method signature to declare that the method may raise a checked exception, warning callers to handle it: `public void withdraw(double amount) throws InsufficientFundsException { ... }` — Without throws in the signature, callers would not know they need to use a try-catch. throw = action; throws = declaration.

51

How do you create a custom exception class called `InsufficientFundsException`?

MODEL ANSWER

`public class InsufficientFundsException extends Exception { public InsufficientFundsException(String message) { super(message); } }` — Extend the Exception class (for a checked exception) or RuntimeException (for unchecked). Pass the message to the parent class using super(). Now you can throw: `throw new InsufficientFundsException("Balance too low");` and catch: `catch (InsufficientFundsException e) { System.out.println(e.getMessage()); }` Custom exceptions make your error-handling code descriptive and domain-specific.

52

Write code to handle multiple possible exceptions in a transaction process using multiple catch blocks.

MODEL ANSWER

`try { int index = Integer.parseInt(userInput); double amount = transactions[index]; processPayment(amount); } catch (NumberFormatException e) { System.out.println("Invalid input: not a number."); } catch (ArrayIndexOutOfBoundsException e) { System.out.println("Invalid transaction index."); } catch (InsufficientFundsException e) { System.out.println("Error: " + e.getMessage()); }` — List more specific exceptions before general ones. Java evaluates catch blocks in order and executes the first matching one. Java 7+ also supports multi-catch: `catch (NumberFormatException | ArrayIndexOutOfBoundsException e).`

53

What classes would you use to read from and write to a text file in Java?

MODEL ANSWER

For writing to a text file, use `FileWriter` or `BufferedWriter` — `BufferedWriter` is preferred as it buffers output for efficiency. For reading, use `FileReader` or `BufferedReader` — `BufferedReader`'s `readLine()` method is convenient. The `java.io` package provides all these classes. In Java 7+, `try-with-resources` ensures files are automatically closed. For higher-level file operations, the `java.nio.file.Files` utility class provides one-liners like `Files.writeString()` and `Files.readString()`.

54

Write code to save a list of transaction logs to a file using `FileWriter`.

MODEL ANSWER

`try (FileWriter fw = new FileWriter("transactions.txt"); BufferedWriter bw = new BufferedWriter(fw)) { for (String log : transactionLogs) { bw.write(log); bw.newLine(); } } catch (IOException e) { System.out.println("File error: " + e.getMessage()); }` — The try-with-resources block auto-closes the writers. `BufferedWriter` wraps `FileWriter` to improve performance. Each log entry is written on its own line using `newLine()`. For appending to an existing file, use `new FileWriter("transactions.txt", true)` with `true` as the append flag.

55

How does file I/O in Java simulate basic data persistence before using a database?

MODEL ANSWER

Without a database, you can persist data by writing it to text or binary files. When the application starts, it reads the file to load saved data into memory; when changes are made, it writes the updated data back. This mimics the load-process-save cycle of a database. For example, a simple expense tracker can store transactions in a CSV file and read it on startup. While files lack querying capabilities, transactions, and concurrent access support, they demonstrate the concept of data persistence that databases formalize.

56

What is the Java Collections Framework, and why is it useful?

Think about storing and managing groups of objects.

MODEL ANSWER

The Java Collections Framework (JCF) is a set of interfaces and classes in `java.util` that provide ready-made data structures for storing and manipulating groups of objects. Key interfaces include `List` (ordered, allows duplicates), `Set` (unordered, no duplicates), `Queue` (FIFO ordering), and `Map` (key-value pairs). Common implementations include `ArrayList`, `HashSet`, and `HashMap`. The JCF saves you from implementing these structures from scratch, is well-tested and optimized, and follows a consistent interface-based design that makes code flexible and reusable.

57

How would you use an `ArrayList` to store a list of transaction objects in Java?

MODEL ANSWER

```
ArrayList<Transaction> transactions = new ArrayList<>(); transactions.add(new Transaction("T001", 500.0)); transactions.add(new Transaction("T002", 200.0)); for (Transaction t : transactions) { System.out.println(t.getId() + " - " + t.getAmount()); } — ArrayList is a dynamic array that grows automatically. You add elements with add(), remove with remove(), and access by index with get(index). It preserves insertion order. Use generics (the <Transaction> part) to make the list type-safe.
```

58

What is the difference between a `List` and a `Set` in Java Collections?

MODEL ANSWER

A `List` is an ordered collection that allows duplicate elements — elements are accessed by index and maintain their insertion order. `ArrayList` and `LinkedList` implement `List`. A `Set` is an unordered collection that does not allow duplicate elements — adding a duplicate simply has no effect. `HashSet` implements `Set` using hashing for fast lookup. Use a `List` when order matters and duplicates are allowed (e.g., transaction history). Use a `Set` when you need uniqueness (e.g., a set of unique user IDs or visited pages).

59

How does a `HashMap` work, and when would you use it to store transaction details?

MODEL ANSWER

A `HashMap` stores data as key-value pairs and uses hashing to enable $O(1)$ average-time retrieval by key. When you put a key-value pair, it hashes the key to find a bucket position. To retrieve, it hashes the key again and goes directly to that bucket. For transaction details, you might use: `HashMap<String, Transaction> txnMap = new HashMap<>(); txnMap.put("T001", transaction1); Transaction t = txnMap.get("T001");` — This gives instant lookup by transaction ID, much faster than searching through a list.

60

Write code to iterate over a HashMap of transaction IDs and print each transaction's details.

MODEL ANSWER

```
for (Map.Entry<String, Transaction> entry : txnMap.entrySet()) { System.out.println("ID: " + entry.getKey() + ", Amount: " + entry.getValue().getAmount()); } — entrySet() gives a set of all key-value pairs. You can also iterate keys only with keySet() or values only with values(). Alternatively, use forEach with a lambda: txnMap.forEach((id, txn) -> System.out.println(id + ": " + txn.getAmount()));
```

61

How would you sort a collection of transaction amounts in Java?

MODEL ANSWER

For a List: Collections.sort(transactionList); sorts in ascending natural order for Comparable types like Double. For reverse order: Collections.sort(transactionList, Collections.reverseOrder()); For sorting a list of custom objects by a field: transactions.sort(Comparator.comparingDouble(Transaction::getAmount)); You can also use: transactions.sort((t1, t2) -> Double.compare(t1.getAmount(), t2.getAmount())); Arrays can be sorted with Arrays.sort(). Always ensure the element type implements Comparable or you provide a Comparator.

62

Explain how keys and values in a Map relate to columns and records in a database.

MODEL ANSWER

In a database table, each row is a record identified by a primary key, and the other columns hold the record's data. A Map mirrors this: the key corresponds to the primary key (like a transaction ID), and the value is the full object (the rest of the record's data). Fetching a record by primary key in SQL — SELECT * FROM transactions WHERE id = 'T001' — is equivalent to txnMap.get('T001') in Java. Both provide fast direct access by identifier. This mapping is why ORM tools like Hibernate use Maps internally to cache entities.

63

What is a thread in Java, and what are the main steps in its lifecycle?

MODEL ANSWER

A thread is the smallest unit of execution within a program — it allows multiple tasks to run concurrently. The thread lifecycle has five states: New (thread object created but not started), Runnable (started and ready to run), Running (actively executing), Blocked/Waiting (paused, waiting for a resource or signal), and Terminated (finished execution). A thread is created by extending Thread or implementing Runnable, and started by calling thread.start(). The JVM's thread scheduler determines which Runnable thread gets CPU time.

64

How would you use a thread to perform a background task, such as updating cashback every 10 seconds?

MODEL ANSWER

```
Thread cashbackThread = new Thread(() -> { while (true) { try { updateCashback(); Thread.sleep(10000); } catch (InterruptedException e) { break; } } }); cashbackThread.setDaemon(true); cashbackThread.start(); — A Runnable lambda defines the task. Thread.sleep(10000) pauses for 10 seconds (10,000 ms). Setting the thread as a daemon ensures it stops automatically when the main program exits. Wrapping the sleep in a try-catch handles the InterruptedException, allowing clean shutdown.
```

65

What is the Runnable interface, and how is it used to create threads in Java?**MODEL ANSWER**

Runnable is a functional interface in `java.lang` with a single method: `public void run()`. Implementing Runnable is the preferred way to define a thread's task because it separates the task logic from the Thread class, allows the class to still extend another class (Java doesn't allow multiple inheritance), and works well with lambda expressions. Example: `Runnable task = () -> System.out.println("Running in background"); Thread t = new Thread(task); t.start();` — You pass the Runnable to a Thread and call `start()` to execute it.

66

How does parallelism using threads improve backend operations in applications like a digital wallet?**MODEL ANSWER**

In a digital wallet backend, many operations can run concurrently rather than sequentially — for example, processing a payment, sending a notification email, and updating cashback rewards do not depend on each other. By running these in separate threads, the total time is closer to the longest single task rather than the sum of all tasks. This improves throughput and responsiveness. Thread pools (via `ExecutorService`) are used in production to manage many concurrent requests efficiently, avoiding the overhead of creating a new thread for every operation.

67

Describe how you would design a console-based Digital Wallet application that allows users to register, log in, check balance, deposit, spend money, and view transaction history.**MODEL ANSWER**

The application would have a main loop displaying a menu. On startup, load existing user data (from a file or in-memory map). For registration, collect name and password, create a User object, and store it in a HashMap keyed by username. For login, verify credentials. On the logged-in menu, route to methods: `checkBalance()` returns the user's balance, `deposit()` adds to it, `spend()` validates sufficient funds and deducts, `addTransaction()` logs each operation to the user's ArrayList of Transaction objects, and `viewHistory()` prints the list. Use exception handling throughout for invalid inputs.

68

How would you use ArrayList or HashMap to temporarily store user data and transactions in your Digital Wallet project?**MODEL ANSWER**

Use a `HashMap<String, User>` to store users by username for fast login lookup. Each User object contains a balance (double) and an `ArrayList<Transaction>` for their transaction history. When a user deposits or spends, add a new Transaction object to their list. When viewing history, iterate the ArrayList and print each entry. The HashMap gives $O(1)$ user lookup, and the ArrayList preserves the chronological order of transactions. This in-memory store is replaced by a database in production.

69

Explain how encapsulation can help secure user credentials in your Digital Wallet application.

MODEL ANSWER

By making the User class's password field private, no external code can directly read or modify it. Access is only through a controlled method like boolean validatePassword(String input) which compares the input against the stored password (ideally hashed). This prevents accidental exposure — other parts of the program can never print or expose the raw password because they have no direct access. Encapsulation also lets you later change how passwords are stored (e.g., switching from plain text to bcrypt hashing) without changing the rest of the codebase.

70

If you had to add a feature to your Digital Wallet app that sends notifications to users when their balance is low, which Java concepts from this module would you use and how would you implement it?

Think about conditional statements, methods, and possibly threads for background checking.

MODEL ANSWER

You would use conditional statements (if balance < threshold) inside a checkAndNotify() method to detect the low balance condition. Encapsulation ensures balance is only checked through methods. A background thread running every few minutes could call this check without blocking the main application. You could use an ArrayList to store a notification log. Exception handling ensures the notification doesn't crash if there is a file write error. In a more complete version, the notification would be sent via file I/O or (later) an email API, applying file I/O concepts from the module.

71

How can you use ChatGPT or GitHub Copilot to help you debug a Java program that is giving a NullPointerException? What steps would you take to verify the suggested fix?

Consider the importance of understanding and testing the AI's suggestion before using it.

MODEL ANSWER

Paste the stack trace and relevant code into ChatGPT or Copilot and ask it to identify where a null dereference could occur. The AI will typically point to the line and explain which variable is null and why. Before applying the fix, read and understand the explanation — don't copy code blindly. Then test by running the specific scenario that caused the NPE. Check for edge cases the AI may have missed (like the variable being null in another code path). Finally, add a null check or initialize the variable properly and confirm the error is resolved with a test run.

72

How could AI tools help you quickly generate boilerplate code for classes or methods in your Java project, and why is it important to review the code before using it?

MODEL ANSWER

AI tools like Copilot can generate full class templates, getters/setters, constructors, and CRUD methods in seconds, saving significant typing time. They are especially useful for repetitive patterns like DAO classes or entity models. However, AI-generated code must be reviewed because it may use deprecated APIs, miss your project's naming conventions, contain logic errors for edge cases, or introduce security vulnerabilities (like not validating inputs). AI is a productivity tool, not a replacement for understanding — you are responsible for the code you ship, so always read, understand, and test what the AI produces.